arXiv:2603.05294v3 [cs.AI] 19 Sep 2026

STRUCTUREDAGENT: Planning with AND/OR Trees for Long-Horizon Web Tasks

Elita A. Lobo
University of Massachusetts Amherst
elobo@umass.edu

Xu Chen
Amazon
frank.chenxu95@gmail.com

Jingjing Meng
Amazon*
jingjing.meng1@gmail.com

Nan Xi
Virginia Commonwealth University
xin@vcu.edu

Yang Jiao
Amazon
jaoyan@amazon.com

Chirag Agarwal
University of Virginia
qze3wn@virginia.edu

Yair Zick
University of Massachusetts Amherst
yzick@umass.edu

Yan Gao
Amazon
yanngao@amazon.com

Abstract

Existing LLM-based web agents struggle on complex, long-horizon tasks due to limited in-context memory, weak planning abilities, and greedy behaviors that lead to premature termination. To address these challenges, we propose STRUCTUREDAGENT, a hierarchical planning framework that interleaves planning and execution via dynamic AND/OR trees. The framework separates structural planning from LLM-based reasoning, enabling principled error recovery through node repair, systematic exploration of alternatives via OR nodes, and modular plans that can facilitate human intervention. On WebArena (630 tasks), STRUCTUREDAGENT achieves a $\sim$53% success rate vs. $\sim$46% for AgentOccam, and on complex multi-constraint Amazon shopping tasks, gains reach 10% over the strongest baseline.

1 Introduction

Recent advances in large language models (LLMs) have enabled agentic systems for sequential decision-making. Such agents must perceive their environment, reason across multiple time steps, and take actions that optimize long-term objectives. A particularly promising application area is the development of web-browsing agents, which navigate and interact with the internet to accomplish complex, goal-driven tasks including information seeking, form filling, and transactional activities. Despite notable progress, building robust and interpretable web-based agents remains challenging.

Although many of these tasks are intuitively straightforward for humans, LLM-powered web agents continue to struggle with complex, multi-step browsing [Yang et al., 2025, Erdogan et al., 2025, Qi et al., 2025]. We focus on agents without specialized memory or retrieval machinery, the prevailing setup among publicly reported web agents. In this setting, four challenges account for the bulk of observed failures: (1) Limited in-context memory, which prevents the agent from tracking alternative solution paths across dense web pages, where a single page can exceed 20k tokens (see Figure 6). Even when long-context models are used, existing agents condition only on the current observation and a summary of the interaction history. (2) Difficulty constructing and

*Work done while at Amazon

Preprint.

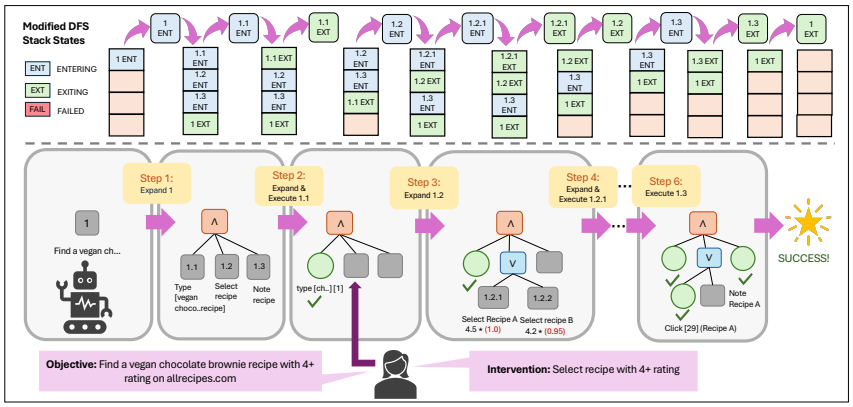


Figure 1: Illustration of StructuredAgent solving a web task via greedy DFS of a dynamically constructed And/Or tree. The root node represents the task objective and is expanded into sub-tasks that are progressively refined and executed. Node types are color-coded to distinguish OR ($\vee$), AND ($\wedge$), and ACTION nodes. The top half tracks the corresponding DFS stack states, where each node enters the stack in one of three states: ENTERING (ENT), EXITING (EXT), or FAILED (FAIL). Node repair and pruning are supported but not triggered in this example, and human intervention is optional.

executing complex plans that must remain coherent over extended horizons. (3) Lack of robust error handling, leaving agents unable to recover from failures or explore alternative solutions [Koh et al., 2025]. (4) Greedy decision-making, which leads to premature task termination before all user constraints are satisfied [Schmied et al., 2025] (see Appendix P).

In cognitive science [Newell et al., 1972], human problem solving is framed as heuristic search coupled with *means-ends analysis*, where complex tasks are recursively decomposed into manageable subproblems. This perspective motivates hierarchical planning in AI, where agents improve efficiency by breaking down complex goals into nested sub-goals and actions at multiple levels of abstraction. Such methods are particularly well-suited to open-ended, web-based tasks characterized by uncertainty and expansive state and action spaces.

Classical hierarchical planners, however, rest on assumptions that web environments violate. A* and AO* require admissible heuristics or fully specified domain models (e.g., PDDL), which are unavailable when state and action spaces are unbounded and only partially observable. Monte Carlo tree search (MCTS) is prohibitively expensive, requiring multiple rollouts before selecting an action at every step. Hierarchical task network (HTN) planners depend on pre-specified decomposition schemas, which complex web tasks rarely provide. The closest applicable works [Koh et al., 2025, Erdogan et al., 2025] either operate at the action level without hierarchical structure or construct linear plans without backtracking.

Recent research has explored explicit planning mechanisms for web agents. Koh et al. [2025] use tree-based planning with A* search to explore alternative solution paths upon failure. However, their method operates at the level of individual actions, which limits exploitation of compositional task structure, and the absence of admissible heuristics can lead to inefficient path switching. Yang et al. [2025] propose incremental tree construction with prune-and-branch strategies, but rely solely on the LLM for plan construction and execution, often resulting in simplistic plans and limited error recovery. Erdogan et al. [2025] introduce dynamic plan revisions, but permit only sequential plans determined prior to execution, without error back-propagation.

To address these challenges, we propose StructuredAgent, a hierarchical planning framework that interleaves planning and execution via ordered And/Or trees. The key architectural choice is a separation of concerns: the framework constructs and maintains the planning tree, while the LLM is invoked only for local operations such as node expansion or repair. Our main contributions are:

- We introduce a hierarchical planning framework enabling inference-time planning via ordered And/Or trees. The framework supports compositional reasoning, dynamic plan revision, and

error backpropagation, while producing modular plans that can facilitate error localization and human intervention.

- We evaluate on WebArena (630 tasks), WebVoyager (129 tasks), and a custom 60-task shopping benchmark, achieving 7–10% improvement over the strongest baseline on complex tasks.

2 Preliminaries

Web-browsing tasks are partially observable and inherently stochastic, as web page content and structure can change dynamically over time, either unpredictably or in response to the agent's actions, and the agent lacks access to the full underlying browser state. The agent is typically limited to the current HTML page, prior actions, and extracted metadata, requiring it to make decisions under uncertainty while maintaining internal beliefs about the environment.

We model the agent-browser interaction as a Partially Observable Markov Decision Process (POMDP) $\langle \mathcal{O}, \mathcal{S}, \mathcal{A}, R, T, p_0, \gamma \rangle$ Kaelbling et al. [1998]. An *observation* $o \in \mathcal{O}$ comprises the task description, agent-constructed plans, interaction metadata and the HTML DOM, $s \in \mathcal{S}$ is the true underlying *state*, and an *action* $a \in \mathcal{A}$ is either an atomic browser operation (click, scroll, type, go back) or a token sequence describing a sub-goal or strategy. The transition kernel $T(s' \mid s, a)$ is induced by the website and unknown to the agent, p_0 is the initial state distribution, and γ discounts future reward. We represent the agent's history-dependent *policy* π using an LLM. Specifically, at timestep t, the observation history $h_t = \{o_1, \ldots, o_t\}$ is encoded as a token sequence $x_t = [x_1, \ldots, x_m]$, and the action $a_t = [y_1, \ldots, y_n]$ is generated autoregressively as $\Pr(a_t \mid h_t) = \prod_{i=1}^{n} \pi(y_i \mid x_1, \ldots, x_m, y_1, \ldots, y_{i-1})$. We provide the full formalization of the problem in Appendix K.

3 StructuredAgent

The central idea of StructuredAgent is to interleave planning and execution via a dynamically constructed AND/OR tree. As new information becomes available, the agent must expand, revise, and prune the planning tree to eliminate unpromising paths and focus search on promising ones.

Rather than delegating full tree management to the LLM, as in AgentOccam [Yang et al., 2025], StructuredAgent manages all tree structure directly, invoking the LLM only for targeted, well-defined operations such as node expansion, repair, completion checking, and pruning. This separation improves reliability and facilitates interpretability.

3.1 AND/OR Planning Tree Structure

The planning tree is composed of three node types: (1) And nodes represent conjunctive sub-goals. All children must succeed for the parent to succeed. Children may be ordered (executed sequentially) or unordered. (2) Or nodes represent alternative strategies. The parent succeeds as soon as any child succeeds. (3) Action nodes are leaves corresponding to concrete browser operations such as typing a query, clicking a button, scrolling, navigating back, or recording a note. We detail the tree operations in the following subsections.

This hierarchical structure allows the agent to decompose tasks into sub-goals and reason explicitly about alternative solution paths. For example, when solving a task such as ‘‘`find a coffee filter under $50`’’, the agent might construct an And node with two children representing searching for "coffee filter" and applying a price filter. Each sub-goal can further decompose or branch into Or nodes that encode different navigation or filtering strategies. This flexibility allows the agent to represent a wide variety of decompositions while capturing the sequential dependencies that arise in web-based workflows.

In Figure 13 we highlight a benefit of our framework's interpretable hierarchical planning structure, namely that it naturally supports human-in-the-loop oversight, whereby an operator can inspect the And/Or tree, detect incorrect sub-goal decompositions, and inject corrective sub-goals at any layer of the plan before execution proceeds further.

3.2 AND/OR Tree Construction and Traversal

Given a web-based task, the goal is to interleave planning and execution so that the agent can continually adapt its strategy as new information becomes available. To achieve this, we use an LLM as a high-level controller that issues localized tree-manipulation directives. These directives determine the type of a node, expand the node, repair it by adding or removing children, or verify whether its objective has been satisfied. The surrounding framework manages tree traversal and executes the corresponding operations.

We use a modified greedy iterative depth-first search (DFS) to expand nodes in STRUCTUREDAGENT, starting at the root node that represents the task objective. When the agent encounters an AND node, it processes its children sequentially according to the node's prescribed ordering. When it encounters an OR node, the agent selects a single child to explore by querying an LLM for an estimate of its likelihood of success based on summaries of prior actions and observations. If the agent reaches an ACTION node, it executes the action in the environment and updates the tree with newly observed information. This update may require pruning or revising certain nodes.

Unlike standard DFS, which operates on a fixed graph, our traversal incrementally constructs and maintains an evolving AND/OR tree whose structure changes as the agent gathers new information. In web-based environments, the search space is large, stochastic, and difficult to model, so sub-goals generated during planning may be imprecise or infeasible, leading to node failures during execution. To handle such failures, STRUCTUREDAGENT supports systematic node revision and, after repeated unsuccessful attempts, node pruning. Pruning a child of an AND node invalidates all subsequent siblings in its prescribed execution order, and these structural changes must propagate upward to preserve global plan coherence, potentially requiring nodes to be revisited multiple times during traversal.

These requirements motivate two modifications to standard DFS. First, we introduce three execution states for nodes in the DFS stack: ENTERING, EXITING, and FAILED. A node enters ENTERING when it has not yet been processed or when some of its children remain unprocessed. It transitions to EXITING once execution completes for ACTION nodes, once all children have been processed for AND nodes, or once the selected child has been processed for OR nodes. In the EXITING state, the framework evaluates whether the node's objective has been satisfied. A node enters FAILED if execution fails for ACTION nodes or if any child fails or is pruned for AND/OR nodes, indicating that revision or pruning is required. These states support upward propagation of structural changes and allow the framework to process a node multiple times as the tree evolves.

Second, classical iterative DFS uses only a binary visited/unvisited distinction, which is insufficient when dynamic pruning and revision require processing a node multiple times. To handle this, we assign each node one of six status values (summarized in Table 28 in the appendix) that capture structural, execution, and failure conditions, enabling the framework to manage the additional complexity introduced by dynamic tree evolution.

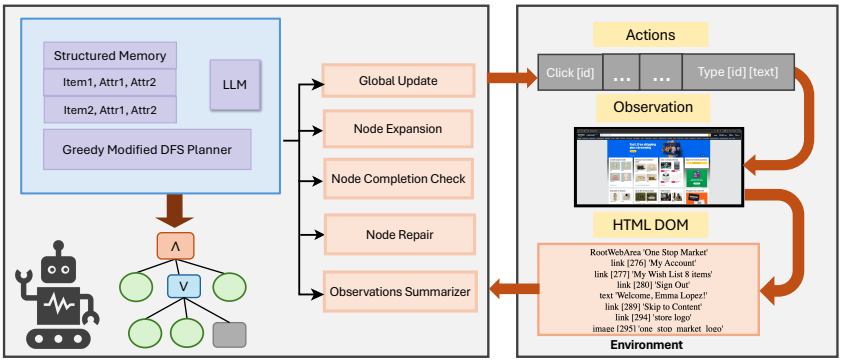


Figure 2: Illustration of the STRUCTUREDAGENT Framework.

We present the high-level tree construction and execution procedure used by STRUCTUREDAGENT in Algorithm 1. We provide an illustration of node state transitions in Figure 11 (appendix). Full

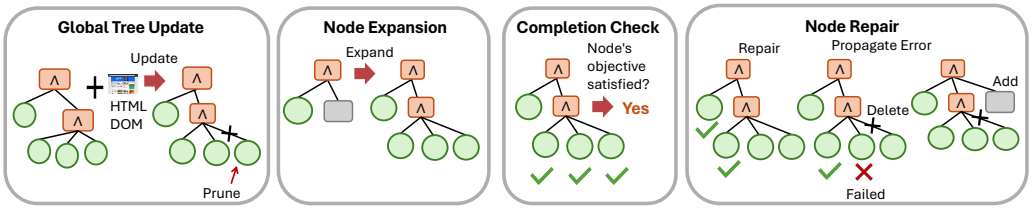


Figure 3: Overview of the primary operations used to construct and maintain the AND/OR tree structure.

details of the planning algorithm are provided in Section O.1. We now describe the fundamental tree operations and the agent modules required by the STRUCTUREDAGENT framework.

3.3 Tree Operations and Agent Modules

We now describe the operations that construct and maintain the AND/OR tree under the guidance of an LLM acting as a high-level controller (illustrated in Figure 3, appendix). Since storing the full observation history in-context is infeasible, the agent maintains concise internal representations of browser interactions as proxies for the full state (detailed in Table 27).

Building on these representations, we introduce five specialized operators that prompt the backbone LLM to perform context-specific inference, with the framework verifying and executing the model's predictions. The NODE EXPANSION operator determines node types and generates children: sub-goals for AND nodes, executable actions for ACTION nodes, and scored candidate strategies for OR nodes. NODE REPAIR revises failed nodes by proposing structural modifications or pruning when recovery is no longer viable. GLOBAL TREE UPDATE maintains tractability by pruning irrelevant branches and refining node descriptions as new context becomes available. NODE COMPLETION CHECK evaluates whether AND nodes have met their objectives by analyzing child outcomes and task summaries. Finally, OBSERVATION SUMMARIZER produces internal representations of task progress and observations, enabling context-aware reasoning. We provide the detailed prompts, inputs, outputs, and further specifications in Appendices G and M.

Equipped with the AND/OR tree and core operators, we now describe the modified greedy, iterative DFS procedure that STRUCTUREDAGENT uses to build, update, and execute its plan. The agent maintains a DFS stack and processes each node in one of three states: ENTERING, EXITING, or FAILED.

When the agent encounters a node in the ENTERING state, it checks whether the node has been expanded. If not, it invokes NODE EXPANSION to determine the node type and generate its children: an ordered list of sub-goals for AND nodes, ranked alternative strategies for OR nodes, or a browser-level action for ACTION nodes. The node is then re-pushed in EXITING and its children are pushed in ENTERING. For AND nodes, all children are pushed in order and for OR nodes, only the highest-ranked unprocessed child is pushed. For ACTION nodes, the agent immediately executes the browser operation, updates its internal context via OBSERVATION SUMMARIZER, and invokes GLOBAL TREE UPDATE to prune irrelevant branches and refine node descriptions.

When the agent processes a node in the EXITING state, it evaluates whether the node has satisfied its objective. An ACTION node is marked successful or failed based on the browser interaction outcome. An AND node succeeds only if all valid children succeed, confirmed via NODE COMPLETION CHECK. An OR node succeeds as soon as any child succeeds. Once marked successful, a node is not revisited.

When the agent encounters a node in the FAILED state, it initiates recovery. If the failed node is an ACTION node, it is pruned. If the failed node is a child of an AND node, remaining unexecuted siblings are also removed since the conjunctive requirement can no longer be met. If an AND node fails, the agent checks its revision budget: if budget remains, it calls NODE REPAIR to add or modify children. However, if the repair is unsuccessful or the budget is exhausted, the node and all descendants are pruned and failure propagates upward. If an OR node fails, the agent selects the next unprocessed child if one exists; otherwise it attempts repair within the revision limit or prunes the node, propagating failure upward.

Algorithm 1: TREE_CONSTRUCTION(root)

```
stack ← [(root, ENTERING)]
counter ← 0
while stack ≠ ∅ and counter < BUDGET do
    (node, state) ← stack.pop()
    if (node.status=SUCCESS ∧ node.rev_cnt≥MAX) ∨ (node.status=DELETED) then continue
    if node.status = PRUNED then
        BACKTRACK_FAILURE(node)
        continue
    if state = ENTERING then
        if node.type = UNKNOWN then node ← NODE EXPANSION(node)
        node.status ← VISITED
        if node.type = ACTION then
            result ← EXECUTE(node)
            OBSERVATION SUMMARIZER(node, result)
            GLOBAL TREE UPDATE(tree)
            stack.push((node, EXITING))
        else if node.type = AND then
            stack.push((node, EXITING))
            foreach child in reverse do stack.push((child, ENTERING))
        else
            stack.push((node, EXITING))
            stack.push((bestChild, ENTERING))
        end
    else if state = EXITING then
        PROCESS_NODE_EXITING(node, stack)
    else if state = FAILED then
        if node.type = ACTION then
            node.status ← PRUNED
            BACKTRACK_FAILURE(node)
        else if node.type = AND then
            if rev_cnt < MAX then node ← NODE REPAIR(node)
            stack.push((node, ENT))
            else node.status ← PRUNED
            BACKTRACK_FAILURE(node)
        else if node.type = OR then
            if ∃ unprocessed child then
                stack.push((node, EXITING))
                stack.push((nextChild, ENTERING))
            else if rev_cnt < MAX then node ← NODE REPAIR(node)
            stack.push((node, ENT))
            else node.status ← PRUNED
            BACKTRACK_FAILURE(node)
        end
    end
end
```

Through iterative expansion, execution, evaluation, and targeted repair, this procedure enables STRUCTUREDAGENT to construct and adapt its hierarchical plan while incorporating new observations and responding to failures. See Appendix O for full algorithmic details.

Example. Consider the task ‘‘`Find a vegan chocolate brownie recipe with 4+ rating on allrecipes.com`’’ (see Figure 2). The agent begins with the root on the stack in the ENTERING state. NODE EXPANSION identifies it as an AND node with three ordered children: search (1.1), select recipe (1.2), and note recipe (1.3). The root is then re-pushed in the EXITING state and its children are pushed in the prescribed order. Node 1.1 then executes and succeeds. Node 1.2 expands into an OR node with two ranked alternatives, but only the top-ranked child 1.2.1 (Recipe A) is pushed. Once it succeeds, NODE COMPLETION CHECK confirms that the OR objective is satisfied, so node 1.2.2 is never explored. OBSERVATION SUMMARIZER updates the internal context after each action, and GLOBAL TREE UPDATE makes no modifications in this trace. After node 1.3 succeeds, the root is popped in the EXITING state, NODE COMPLETION CHECK confirms that all three AND children succeeded, and the agent terminates with SUCCESS. No node enters the FAILED state here. Had node 1.2 exhausted both alternatives, the sibling node 1.3 would have been pruned and the failure would have propagated to the root, invoking NODE REPAIR if the revision budget permitted. We provide the full step-by-step trace in Appendix L.

Method	Map ↑ (n=128)	Shopping ↑ (n=192)	Reddit ↑ (n=114)	GitLab ↑ (n=196)	Overall ↑ (n=630)
StructuredAgent†	0.45 [0.37, 0.53]	**0.53 [0.46, 0.60]**	**0.73 [0.64, 0.80]**	0.46 [0.39, 0.53]	**0.53 [0.49, 0.56]**
AgentOccam (Claude)	**0.46 [0.38, 0.55]**	0.38 [0.31, 0.45]	0.60 [0.51, 0.69]	**0.47 [0.40, 0.54]**	0.46 [0.43, 0.50]
AgentOccam (GPT-4o)	0.45 [0.36, 0.53]	0.35 [0.29, 0.42]	0.62 [0.54, 0.71]	0.33 [0.26, 0.40]	0.41 [0.38, 0.45]
SteP (GPT-4o)	0.33 [0.26, 0.41]	0.27 [0.21, 0.33]	0.56 [0.47, 0.65]	0.22 [0.17, 0.28]	0.32 [0.29, 0.36]
BasicClaudeAction	0.37 [0.29, 0.45]	0.30 [0.24, 0.36]	0.30 [0.22, 0.39]	0.28 [0.22, 0.34]	0.31 [0.28, 0.34]
WebarenaReplication	0.31 [0.24, 0.39]	0.31 [0.25, 0.38]	0.20 [0.13, 0.27]	0.29 [0.23, 0.35]	0.28 [0.25, 0.32]

Table 1: Success rates on WebArena across four task categories, as mean [95% bootstrap CI]. **Bold** is best per column.
† $p < 0.05$ vs. all methods (one-sided permutation test).

4 Related Work

LLM Inference-Based Web Agents: Several works [Putta et al., 2025, Zhang et al., 2024b, He et al., 2024, Zhou et al., 2024a] leveraged closed-source LLMs or multimodal models for complex web tasks via inference-time strategies. Wang et al. [2025] extracted reusable patterns from prior trajectories to guide navigation, while others employed Monte Carlo Tree Search [Zhang et al., 2024b] or value networks [Koh et al., 2024] for exploration and backtracking. Reflexion [Shinn et al., 2023] added self-reflection, often at the cost of frequent resets. Hierarchical planning has also been explored in this setting. Erdogan et al. [2025] proposed an optimization layer for higher-level planning, and Sodhi et al. [2024] proposed dynamic multi-level control that relies on domain-specific prompts. Yang et al. [2025] introduced a lightweight tree with text filtering and incremental revision, but their agent struggles on complex tasks due to limited error recovery. In contrast, our AND/OR planning tree supports robust error handling and explicit dependency tracking, and generates modular trajectories (see Table 32).

Fine-Tuned LLMs as Web Agents: A complementary line of work fine-tunes open-weight models on expert trajectories via imitation or reinforcement learning [Lai et al., 2024, Hong et al., 2024, Zhang et al., 2024a, Murty et al., 2025, Pareja et al., 2024, Prasad et al., 2024, Bai et al., 2024, Zhou et al., 2024c, Wang et al., 2024]. These approaches face challenges such as brittleness under distribution shift and the cost of collecting expert demonstrations for imitation learning. We expand on this discussion in Appendix A.

5 Experiments

Setup. We evaluate StructuredAgent on three benchmarks. The first is an Amazon dataset of 100 tasks, which combines the 40 Amazon tasks from WebVoyager with 60 custom tasks that each impose multiple constraints. The second, WebVoyager-Other, consists of the remaining 89 WebVoyager tasks. The third is WebArena, where we evaluate on 630 tasks from the original benchmark and on WebArena-Verified, an audited release that reviews every task, reference answer, and evaluator to remove brittle string matching and ambiguous success criteria. All experiments use Claude 3.5 Sonnet as the backbone unless noted otherwise. We compare against AgentOccam [Yang et al., 2025], which uses LLM-driven prune-and-branch planning, and BasicClaudeAction, a flat agent that conditions on the full action history. On WebArena, we additionally include SteP [Sodhi et al., 2024] and the reference WebArena agent (WebarenaReplication). We measure task completion (success rate) using three LLM judges (GPT-4.1, Kimi-k2, Gemini-2.5-Flash) for WebVoyager and Amazon tasks and the official code-based evaluators for WebArena. We further collect human annotations on a subset of Amazon tasks. We provide full details of the setup in Appendix B.1.

5.1 Results

RQ1) How does StructuredAgent compare with LLM-based agents on short-horizon and complex long-horizon tasks? Table 2 reports success rates on Amazon and WebVoyager-Other tasks using Claude 3.5 Sonnet as the backbone. StructuredAgent achieves the highest average performance on Amazon tasks, outperforming AgentOccam by 10 points. On WebVoyager-Other, StructuredAgent and AgentOccam fall within 1.5 points of each other (78–80%), suggesting

Backbone	Method	Amazon				WebVoyager Other			
		kimi	Gemini	GPT	Avg	kimi	Gemini	GPT	Avg
C3.5	StructuredAgent	**62.0**	**47.0**	55.0	**54.7**	80.9	**73.0**	80.9	78.3
	AgentOccam	52.0	35.0	47.0	44.7	**84.3**	71.9	**83.2**	**79.8**
	BasicClaudeAction	48.0	35.0	45.0	42.7	74.2	67.4	70.8	70.8
	SteP	33.0	29.0	33.0	31.6	74.2	60.7	69.7	68.2
C3.7	StructuredAgent	**56.7**	**36.7**	**46.7**	**46.7**				
	AgentOccam	45.0	30.0	41.7	38.9	*not evaluated*			
	BasicClaudeAction	28.3	21.7	20.0	23.3				

Table 2: Success rates (%) on Amazon shopping tasks and WebVoyager Other under three LLM judges (Kimi-K2, Gemini-2.5-Flash, GPT-4.1), with Claude 3.5 (C3.5) and Claude 3.7 (C3.7) backbones. StructuredAgent leads on Amazon by 10 points (C3.5) and 7.8 points (C3.7); on WebVoyager Other, StructuredAgent and AgentOccam fall within 1.5 points (78–80%).

Site (n)	AO	SA	p
Shopping (56)	35.7	**39.3**	.422
Shop. Admin (53)	11.3	**15.1**	.386
GitLab (56)	30.4	**51.8**	.016*
Reddit (42)	66.7	**76.2**	.235
Cross-site (48)	0.0	0.0	1.00
Overall (255)	27.8	**35.7**	**.037***

Table 3: WebArena-Verified Hard. StructuredAgent leads AgentOccam by 7.8 points overall on 255 tasks ($p = 0.037$).

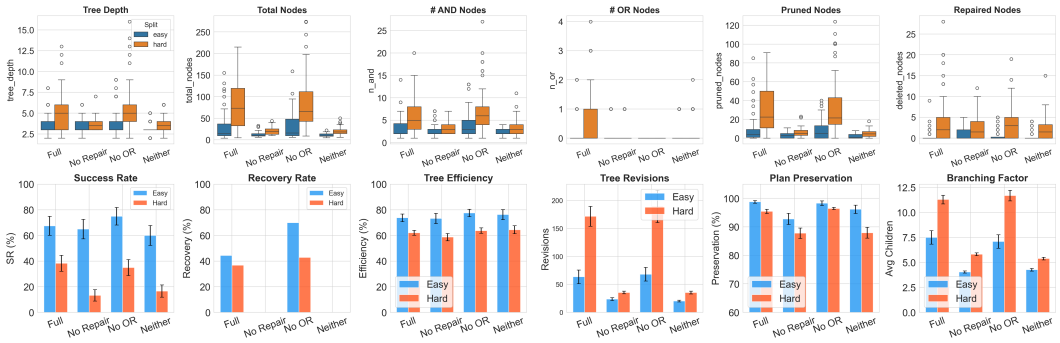
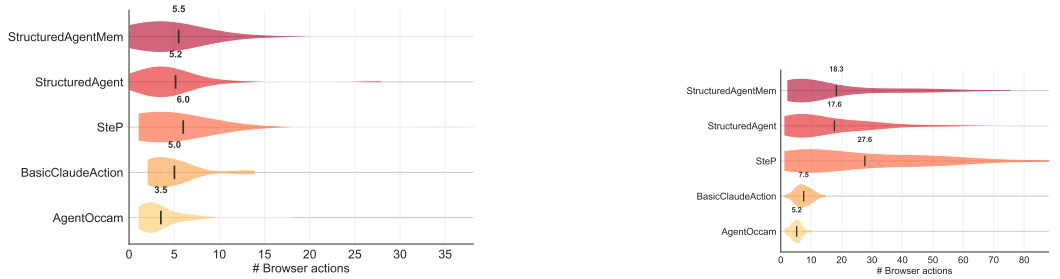


Figure 4: Ablation analysis on Amazon tasks. Top: tree structure metrics (depth, nodes, pruning). Bottom: success, repair trigger, recovery rate, and revisions. The full agent (Repair+OR) builds deeper trees and achieves non-zero recovery on hard tasks.



(a) Amazon-easy

(b) Amazon-hard

Figure 5: Trajectory length distributions on Amazon tasks. On hard tasks, StructuredAgent explores more strategies before terminating, leading to higher success rate, while baselines terminate prematurely with shorter but less successful trajectories.

that these short-horizon tasks do not discriminate between planning approaches. The value of hierarchical planning emerges only on the harder Amazon tasks, which require tracking multiple candidate items under several simultaneous constraints.

Table 1 reports results across WebArena task categories. StructuredAgent achieves the highest overall performance (53%), surpassing AgentOccam by 7%, with the largest gains on Shopping (+15%) and Reddit (+13%), the categories with the most complex multi-step workflows. On Map and GitLab, which involve simpler navigation patterns, StructuredAgent performs comparably to AgentOccam. Hierarchical planning helps most where task complexity is highest.

RQ2) Does STRUCTUREDAGENT continue to benefit from hierarchical planning with a stronger backbone? Table 2 compares performance on Amazon tasks using Claude 3.7. Despite Claude 3.7 being explicitly trained on agentic tasks, STRUCTUREDAGENT still outperforms AgentOccam by 7.8% and BasicClaudeAction by 23.4%, demonstrating that the benefits of hierarchical planning persist with stronger models.

RQ3) Does STRUCTUREDAGENT generalize to model families beyond Claude? Table 6 evaluates STRUCTUREDAGENT and AgentOccam using Kimi-k2-0905 as the backbone on a subset of WebArena tasks. STRUCTUREDAGENT achieves overall performance comparable to AgentOccam but does not outperform it, leading only on the Shopping category. We attribute this to the weaker reasoning and planning capabilities of Kimi-k2. STRUCTUREDAGENT's gains depend on the backbone's ability to produce good decompositions and rank alternatives reliably, and Kimi-k2 is weaker at both, so the tree provides less useful guidance than with Claude.

RQ4) How does STRUCTUREDAGENT compare in terms of trajectory length? We compare STRUCTUREDAGENT and AgentOccam's trajectory lengths on 40 Amazon tasks from WebVoyager, which we label *Amazon-easy*, and 60 Amazon tasks that we generate, each combining multiple constraints, which we label *Amazon-hard* (Figure 5). STRUCTUREDAGENT takes a comparable number of actions to AgentOccam on Amazon-easy, while on Amazon-hard it produces markedly longer trajectories as it explores multiple strategies before terminating. This systematic exploration is precisely what drives its higher success rate. On WebArena, STRUCTUREDAGENT matches AgentOccam on Reddit and Shopping while achieving substantially higher success rates, and takes moderately more actions on Map and GitLab, reflecting increased exploration once straightforward strategies fail. We report per-category WebArena distributions in Appendix B (Figure 9) and demonstrate the exploration behavior with a qualitative example in Table 33.

RQ5) How does STRUCTUREDAGENT perform on WebArena-Verified Hard (a curated difficult subset)? We evaluate on WebArena-Verified Hard [hattami et al., 2025], a 255-task subset selected via difficulty modeling and equipped with deterministic, human-verified evaluators that eliminate false positives and negatives. Table 3 shows STRUCTUREDAGENT outperforms AgentOccam by 21 points on GitLab and 10 points on Reddit, the two categories with the most complex multi-step workflows, and by 7.8 points overall ($p = 0.037$). Hierarchical planning provides the largest benefit on genuinely hard tasks.

RQ6) Which components of STRUCTUREDAGENT drive the performance gains? To isolate the contribution of each component, we disable the two adaptive mechanisms of the AND/OR planner, NODE REPAIR and OR alternatives, individually and jointly on the Amazon-easy (40 tasks) and Amazon-hard (60 tasks) splits with a Haiku 4.5 backbone. This yields four conditions. Full enables both mechanisms, Repair only retains NODE REPAIR but disables OR alternatives, OR only retains OR alternatives but disables repair, and Neither disables both, reducing the planner to a static tree that can neither recover from failed nodes nor fall back to alternative paths. Alongside success rate, we track tree statistics that capture plan structure, adaptation, and recovery (full definitions in Appendix D).

Results show that NODE REPAIR drives recovery, which we measure as the fraction of repair attempts that restore a failed node to a passing state. OR only and Neither cannot recover failed nodes. With repair enabled, the Full planner recovers 44% of failed nodes on Amazon-easy and 37% on Amazon-hard, and Repair only reaches 70% and 43% respectively. Repair also enables deeper exploration. On Amazon-hard, the Full planner builds trees of 80 nodes at depth 5.3 with 172 revisions (expansions, repairs, and prunings), compared to 20 nodes at depth 3.6 without repair. At the same time, repair modifies only a small part of the plan. Plan Preservation, the fraction of nodes left intact by repair, stays between 93 and 99% on Amazon-easy and between 88 and 96% on Amazon-hard, so recovery reuses most of the existing plan rather than replanning from scratch. OR nodes are exercised rarely on these tasks (mean count at most 0.6 across conditions), so node-level repair carries most of the recovery behavior here. Consistent with this, enabling OR alternatives without repair provides no reliable benefit over disabling both mechanisms. On Amazon-hard, OR only scores 13.3% versus 16.7% for Neither under the Gemini judge, and the two conditions perform comparably on Amazon-easy. Without repair, the additional branching goes unexploited, since failed alternatives cannot be restored. OR alternatives matter more when the agent must explore and track multiple candidate paths before arriving at the correct solution. We report per-condition tree statistics in Tables 7 and 8.

Table 4: STRUCTUREDAGENT vs. AgentOccam diagnosis quality on the 15 shared failures; cells give % favoring STRUCTUREDAGENT (raw SA/AO/Tie). Details in Table 9.

Dimension	Gemini 2.5 Flash	Opus 4.8
Precision	86.8 (203/31/3)	94.7 (160/9/70)
Self sufficiency	64.3 (9/5/223)	72.4 (155/59/25)
Clarity	63.2 (91/53/93)	76.9 (183/55/1)
Actionability	78.3 (162/45/30)	67.6 (148/71/20)
Overall preference	81.7 (192/43/2)	75.2 (170/56/13)

Additional analyses. Due to space constraints, we defer to the appendix our study of agreement among the judges and with a human annotator (Table 11). Table 4 summarizes how the framework helps identify the cause of a failure.

6 Conclusion

We presented STRUCTUREDAGENT, a hierarchical planning framework that decomposes complex web tasks into dynamic AND/OR trees, separating structural planning from LLM-based reasoning. The framework enables principled error recovery via node repair, systematic exploration of alternatives via OR nodes, and interpretable plans that support human intervention. On WebArena (630 tasks) and WebVoyager augmented with 60 complex shopping tasks, STRUCTUREDAGENT consistently outperforms strong baselines across multiple model families, with gains of 7–10% over AgentOccam on complex tasks. Ablations confirm that both repair and OR-node mechanisms contribute complementary benefits.

References

Mohamed Aghzal, Gregory J. Stein, and Ziyu Yao. Why do llm-based web agents fail? a hierarchical planning perspective, 2026. URL https://arxiv.org/abs/2603.14248.

Sagnik Anupam, Davis Brown, Shuo Li, Eric Wong, Hamed Hassani, and Osbert Bastani. Browserarena: Evaluating llm agents on real-world web navigation tasks, 2025. URL https://arxiv.org/abs/2510.02418.

Sagnik Anupam, Davis Brown, Shuo Li, Eric Wong, Hamed Hassani, and Osbert Bastani. Browserarena: Evaluating LLM agents on real-world web navigation tasks, 2026. URL https://openreview.net/forum?id=lmeXa6aaoR.

Hao Bai, Yifei Zhou, Mert Cemri, Jiayi Pan, Alane Suhr, Sergey Levine, and Aviral Kumar. Digirl: Training in-the-wild device-control agents with autonomous reinforcement learning. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems*, volume 37, pages 12461–12495. Curran Associates, Inc., 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024/file/1704ddd0bb89f159dfe609b32c889995-Paper-Conference.pdf.

Lutfi Eren Erdogan, Nicholas Lee, Sehoon Kim, Suhong Moon, Hiroki Furuta, Gopala Anumanchipalli, Kurt Keutzer, and Amir Gholami. Plan-and-act: Improving planning of agents for long-horizon tasks, 2025. URL https://arxiv.org/abs/2503.09572.

Amine El hattami, Megh Thakkar, Nicolas Chapados, and Christopher Pal. Webarena verified: Reliable evaluation for web agents. In *Workshop on Scaling Environments for Agents*, 2025. URL https://openreview.net/forum?id=94tlGxmqkN.

Hongliang He, Wenlin Yao, Kaixin Ma, Wenhao Yu, Yong Dai, Hongming Zhang, Zhenzhong Lan, and Dong Yu. WebVoyager: Building an end-to-end web agent with large multimodal models. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6864–6890, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.371. URL https://aclanthology.org/2024.acl-long.371/.

Wenyi Hong, Weihan Wang, Qingsong Lv, Jiazheng Xu, Wenmeng Yu, Junhui Ji, Yan Wang, Zihan Wang, Yuxuan Zhang, Juanzi Li, Bin Xu, Yuxiao Dong, Ming Ding, and Jie Tang. Cogagent: A visual language model for gui agents, 2024. URL https://arxiv.org/abs/2312.08914.

Leslie Pack Kaelbling, Michael L Littman, and Anthony R Cassandra. Planning and acting in partially observable stochastic domains. *Artificial intelligence*, 101(1-2):99–134, 1998.

Jing Yu Koh, Robert Lo, Lawrence Jang, Vikram Duvvur, Ming Lim, Po-Yu Huang, Graham Neubig, Shuyan Zhou, Russ Salakhutdinov, and Daniel Fried. VisualWebArena: Evaluating multimodal agents on realistic visual web tasks. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 881–905, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.50. URL https://aclanthology.org/2024.acl-long.50/.

Jing Yu Koh, Stephen Marcus McAleer, Daniel Fried, and Russ Salakhutdinov. Tree search for language model agents, 2025. URL https://openreview.net/forum?id=kpL66Mvd2a.

Hanyu Lai, Xiao Liu, Iat Long Iong, Shuntian Yao, Yuxuan Chen, Pengbo Shen, Hao Yu, Hanchen Zhang, Xiaohan Zhang, Yuxiao Dong, and Jie Tang. Autowebglm: A large language model-based web navigating agent. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '24, page 5295–5306, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400704901. doi: 10.1145/3637528.3671620.

Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. Offline reinforcement learning: Tutorial, review, and perspectives on open problems, 2020. URL https://arxiv.org/abs/2005.01643.

William Lugoloobi, Samuelle Marro, Jabez Magomere, Joss Wright, and Chris Russell. Known by their actions: Fingerprinting llm browser agents via ui traces, 2026. URL https://arxiv.org/abs/2605.14786.

Shikhar Murty, Hao Zhu, Dzmitry Bahdanau, and Christopher D. Manning. Nnetnav: Unsupervised learning of browser agents through environment interaction in the wild, 2025. URL https://arxiv.org/abs/2410.02907.

Ofir Nachum, Mohammad Norouzi, Kelvin Xu, and Dale Schuurmans. Bridging the gap between value and policy based reinforcement learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, page 2772–2782. Curran Associates Inc., 2017. ISBN 9781510860964.

Allen Newell, Herbert Alexander Simon, et al. *Human problem solving*, volume 104. Prentice-hall Englewood Cliffs, NJ, 1972.

Aldo Pareja, Nikhil Shivakumar Nayak, Hao Wang, Krishnateja Killamsetty, Shivchander Sudalairaj, Wenlong Zhao, Seungwook Han, Abhishek Bhandwaldar, Guangxuan Xu, Kai Xu, Ligong Han, Luke Inglis, and Akash Srivastava. Unveiling the secret recipe: A guide for supervised fine-tuning small llms, 2024. URL https://arxiv.org/abs/2412.13337.

Archiki Prasad, Alexander Koller, Mareike Hartmann, Peter Clark, Ashish Sabharwal, Mohit Bansal, and Tushar Khot. ADaPT: As-needed decomposition and planning with language models. In Kevin Duh, Helena Gomez, and Steven Bethard, editors, *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 4226–4252, Mexico City, Mexico, June 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-naacl.264. URL https://aclanthology.org/2024.findings-naacl.264/.

Pranav Putta, Edmund Mills, Naman Garg, Sumeet Ramesh Motwani, Elan Sopher Markowitz, Julia Kiseleva, Chelsea Finn, Divyansh Garg, and Rafael Rafailov. Agent q: Advanced reasoning and learning for autonomous AI agents, 2025. URL https://openreview.net/forum?id=LuytzzohTa.

Zehan Qi, Xiao Liu, Iat Long Iong, Hanyu Lai, Xueqiao Sun, Jiadai Sun, Xinyue Yang, Yu Yang, Shuntian Yao, Wei Xu, Jie Tang, and Yuxiao Dong. WebRL: Training LLM web agents via self-evolving online curriculum reinforcement learning. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=oVKEAFjEqv.

Thomas Schmied, Jörg Bornschein, Jordi Grau-Moya, Markus Wulfmeier, and Razvan Pascanu. Llms are greedy agents: Effects of rl fine-tuning on decision-making abilities, 2025. URL https://arxiv.org/abs/2504.16078.

Orit Shahnovsky and Rotem Dror. Ai planning framework for llm-based web agents, 2026. URL https://arxiv.org/abs/2603.12710.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning, 2023. URL https://arxiv.org/abs/2303.11366.

Max Simchowitz, Daniel Pfrommer, and Ali Jadbabaie. The pitfalls of imitation learning when actions are continuous, 2025. URL https://arxiv.org/abs/2503.09722.

Paloma Sodhi, S.R.K Branavan, Yoav Artzi, and Ryan McDonald. Step: Stacked LLM policies for web actions. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=5fg0VtRxgi.

Aram Vardanyan. Building browser agents: Architecture, security, and practical solutions, 2025. URL https://arxiv.org/abs/2511.19477.

Huaijie Wang, Shibo Hao, Hanze Dong, Shenao Zhang, Yilin Bao, Ziran Yang, and Yi Wu. Offline reinforcement learning for llm multi-step reasoning, 2024. URL https://arxiv.org/abs/2412.16145.

Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory, 2025. URL https://openreview.net/forum?id=PfYg3eRrNi.

Ke Yang, Yao Liu, Sapana Chaudhary, Rasool Fakoor, Pratik Chaudhari, George Karypis, and Huzefa Rangwala. Agentoccam: A simple yet strong baseline for LLM-based web agents. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=oWdzUpOlkX.

Tao Yu, Zhengbo Zhang, Zhiheng Lyu, Junhao Gong, Hongzhu Yi, Xinming Wang, Yuxuan Zhou, Jiabing Yang, Ping Nie, Yan Huang, and Wenhu Chen. Browseragent: Building web agents with human-inspired web browsing actions, 2025. URL https://arxiv.org/abs/2510.10666.

Maryam Zare, Parham M. Kebria, Abbas Khosravi, and Saeid Nahavandi. A survey of imitation learning: Algorithms, recent developments, and challenges, 2023. URL https://arxiv.org/abs/2309.02473.

Jiwen Zhang, Jihao Wu, Teng Yihua, Minghui Liao, Nuo Xu, Xiao Xiao, Zhongyu Wei, and Duyu Tang. Android in the zoo: Chain-of-action-thought for GUI agents. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen, editors, *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 12016–12031, Miami, Florida, USA, November 2024a. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-emnlp.702. URL https://aclanthology.org/2024.findings-emnlp.702/.

Yao Zhang, Zijian Ma, Yunpu Ma, Zhen Han, Yu Wu, and Volker Tresp. Webpilot: A versatile and autonomous multi-agent system for web task execution with strategic exploration, 2024b. URL https://arxiv.org/abs/2408.15978.

Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning acting and planning in language models, 2024a. URL https://arxiv.org/abs/2310.04406.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents, 2024b. URL https://arxiv.org/abs/2307.13854.

Yifei Zhou, Andrea Zanette, Jiayi Pan, Sergey Levine, and Aviral Kumar. ArCHer: Training language model agents via hierarchical multi-turn RL. In *Forty-first International Conference on Machine Learning*, 2024c. URL https://openreview.net/forum?id=b6rA0kAHT1.

STRUCTUREDAGENT: Planning with AND/OR Trees for Long-Horizon Web Tasks

Supplementary Material

A Extended Related Work

We provide an extended discussion of fine-tuned LLMs as web agents, complementing the Related Work section in the main paper.

Fine-Tuned LLMs as Web Agents: Several works employ open-source language models as web agents [Hong et al., 2024, Lai et al., 2024, Zhang et al., 2024a]. These models often fail because they are small and lack domain-specific knowledge. Follow-up work addresses this by fine-tuning open-source models on expert trajectories collected from human demonstrators or proprietary LLMs [Murty et al., 2025, Pareja et al., 2024, Prasad et al., 2024]. Fine-tuning yields meaningful gains, particularly when the evaluation tasks come from the same distribution as the training data.

Training in this line of work follows one of two paradigms. Imitation learning trains agents offline on pre-collected expert demonstrations [Zare et al., 2023], while reinforcement learning (RL) improves agents through direct interaction with the environment [Bai et al., 2024, Zhou et al., 2024c, Wang et al., 2024]. Imitation learning maximizes the likelihood of expert actions, which leaves agents brittle on complex or out-of-distribution tasks [Simchowitz et al., 2025]. Its success also depends on the quality and breadth of the demonstration data, which is costly to obtain, especially when proprietary models generate the trajectories.

RL offers a path toward more adaptive agents but brings its own challenges [Levine et al., 2020]. Online RL for web agents is computationally demanding, and the reward is typically a sparse binary signal tied to overall task completion. This sparsity creates a credit assignment problem [Nachum et al., 2017], since a failed task can penalize intermediate actions that were in fact correct. Fine-tuning capable and generalizable web agents with high-quality trajectories and dense reward signals therefore remains underexplored.

B Experimental Details

B.1 Experimental Setup

Benchmark Details. *Custom Complex Shopping* contains 60 long-horizon Amazon shopping tasks. Each task is described by a natural language query that encodes detailed constraints over one or more products, and the agent must produce a set of recommendations satisfying these constraints. Task completion typically requires 10 to 30 interaction steps, reflecting the compositional and long-horizon nature of the goals. *WebVoyager* [He et al., 2024] provides real-world tasks on diverse live websites (*e.g.,* BBC News, arXiv, Amazon, GitHub, and HuggingFace). We use a curated subset of 129 tasks. We exclude tasks that require login credentials or otherwise raise security concerns, since the agent cannot reliably bypass them in our evaluation setting. *WebArena* [Zhou et al., 2024b] contains 812 tasks, of which we evaluate on 630. We exclude tasks on the `ShoppingAdmin` site as well as tasks where agents consistently fail due to website errors or evaluator failures, which prevent reliable comparison.

Evaluation Details. For WebVoyager, the LLM judge first decomposes the task into item-level and task-level constraints and then awards one point per satisfied constraint. A constraint counts as satisfied only when its satisfaction is explicitly supported by evidence extracted from the raw observations (Table 16, with the full prompt in Section G). Prior works evaluate agents from the last N observations using a GPT-4V judge. This approach is insufficient for long-horizon tasks, where the required N can be very large, and the prompt used is overly simplistic. For WebArena, we use the benchmark's official code-based (HTML- and URL-based) evaluators without modification.

Baseline Details. We compare against four baselines. AgentOccam [Yang et al., 2025] is a planning agent that incrementally constructs and prunes an action tree. BasicClaudeAction augments a

Table 5: Decoding configuration by model backend. The Anthropic (Bedrock), Novita, and Gemini APIs do not accept a request seed, so determinism on those paths follows from greedy decoding. A request seed (fixed to 0) is honored only on the OpenAI path.

Backend	**Temp.**	**top-p**	**Seed**
Claude (Bedrock `invoke_model`)	0.0	NA	NA
Bedrock Converse (Kimi/Qwen/Nova/...)	0.0	NA	NA
OpenAI GPT	0.0	0.01	0
Kimi (Novita)	0.1	0.95	NA
Gemini	0.0	NA	NA

Claude-based agent with access to the full action history as additional context. SteP [Sodhi et al., 2024] is a stack-based agent that uses human-written atomic strategies designed for WebArena-style tasks. WebarenaReplication is the reference agent released with WebArena, prompted to perform chain-of-thought reasoning. We use the implementations of SteP and WebarenaReplication provided by Yang et al. [2025].

Decoding Configuration. Table 5 summarizes the decoding settings used for each model backend.

Statistical Inference. For WebArena-Verified Hard we report results on the 255 valid-shared tasks. Confidence intervals use the percentile bootstrap. For each site and for the overall score, we resample the binary per-task scores with replacement ($B = 10{,}000$ resamples, each of size equal to the number of tasks), take the mean of each resample, and report the 2.5th and 97.5th percentiles as the 95% CI. Significance uses a one-sided permutation test of SA $>$ AO. We pool both agents' per-task scores, randomly reassign the agent labels $10{,}000$ times, and report as p the fraction of permuted mean differences that meet or exceed the observed difference. Both procedures use a fixed random seed of 42.

Amazon Shopping Tasks

- Recommend 3 standing desks under $350 that support lift ranges from ≤ 24" to ≥ 48" and include a cable management tray. Each recommendation should differ in brand or surface finish. Do not add to cart–list lift range, tray type, and price.
- Recommend 3 pressure cookers under $100 with stainless steel inner pots and yogurt mode. Each must be from a different brand. Do not add to cart–list material, features, rating, and price.
- Recommend 3 wireless printers under $150 that support both AirPrint and borderless 4x6 photo printing. Do not add to cart–list brand, print support, and price.
- Recommend 3 mechanical keyboards under $400 with hot-swappable keys. Ensure variety in switch type or layout (e.g., 75%, TKL). Do not add to cart–list hot-swap support, switch type, and price.
- Recommend 3 laptop backpacks under $200 that support 17-inch laptops, include a padded sleeve, and have a hidden anti-theft pocket. Ensure variation in material, port support, or design. Do not add to cart–list capacity, features, and price.
- Recommend 3 noise-canceling headphones under $300 with multi-device pairing and passive listening support. Ensure brand diversity. Do not add to cart–list battery specs, pairing support, and price.
- Recommend 3 air purifiers under $350 with washable pre-filters and a night mode that disables display lights. Do not add to cart–list pre-filter type, noise level, and price.
- Recommend 3 external SSDs (1TB) under $120 that use USB-C and offer hardware encryption. Ensure diversity in brand or ruggedness. Do not add to cart–list encryption support, interface, and price.
- Recommend 3 espresso machines under $450 that include a milk frother and removable water tank. Ensure different frother types or build designs. Do not add to cart–list milk system, tank size, and price.

- Recommend 3 portable projectors under $300 with 1080p native resolution, tripod screw mount, and built-in Bluetooth speakers. Ensure model diversity. Do not add to cart–list resolution, mounting, audio features, and price.
- Find the cheapest projector and screen set under $450: (1) portable projector with native 1080p, Bluetooth 5.0+, keystone correction, and 300+ ANSI lumens; (2) 70–100 inch outdoor screen. Recommend 3 combinations varying by projector brand. Choose the set with the highest total reviews. List model names, Bluetooth version, brightness, and total price.
- Find the cheapest pair under $350 that includes: (1) over-ear wireless ANC headphones with 40h+ battery life and USB-C charging; (2) premium Bluetooth 5.0+ speaker with stereo pairing and IPX5 water resistance. Both must be rated 4.3+ stars. Recommend 3 brand-distinct combos. List battery specs, pairing and water-resistance features, rating, and total cost.
- Recommend 3 lightweight work kits under $400 with fast delivery: (1) adjustable aluminum laptop stand supporting 10kg+; (2) rechargeable wireless keyboard + mouse; (3) USB-C hub with HDMI, SD, and Ethernet. Choose the combo with the fastest shipping (2 days or less). List ports, delivery times, and prices.
- Under $450, find the most-reviewed premium travel set: (1) 17 inch anti-theft backpack with TSA lock; (2) 8-piece compression packing cube set made from water-resistant fabric; (3) digital luggage scale with auto-off, tare, and backlight. All rated 4.3+ stars. Recommend most-reviewed option. List features, review counts, and total price.
- Recommend the cheapest high-end ergonomic combo under $900: (1) dual-motor standing desk (dual 275+ lb capacity, cable tray, programmable presets); (2) office chair with adjustable headrest, and lumbar support. Both must be rated 4.5+ stars and have 500+ reviews. List brands, specs, and combined price.
- Find the fastest-delivery smart home bundle under $500: (1) smart speaker with built-in voice assistant and premium audio; (2) smart bulb pack with color plus tuneable white and 10K+ hour lifespan; (3) smart plug with energy monitoring and USB port. All must be rated 4.5+ stars and offer two-day delivery. List assistant, bulb specs, plug features, delivery ETA, and price.
- Recommend 3 pro-level chef's prep kits under $400: (1) 8" stainless chef's knife with full tang and thermo-transfer handle; (2) digital food thermometer with $\pm 0.1°$C accuracy, fast-read (<5s), and auto-off; (3) bamboo cutting board set with juice groove and non-slip feet. Choose highest-rated combination. List specs, ratings, and price.
- Find the most-reviewed outdoor adventure set under $500 each: (1) waterproof hiking backpack (30–40L) with rain cover; (2) rechargeable headlamp (500+ lumens) with adjustable beam; (3) vacuum-insulated stainless steel bottle (32 oz) with sweat-proof exterior. All rated 4.5+ stars. List capacities, feature specs, review counts, and total price.
- Recommend 3 advanced artist kits under $250: (1) LED desk lamp with CRI> 90 and adjustable 3000K–6500K color; (2) sketchbook A4 with 200gsm acid-free paper; (3) graphite pencil set including 2H–8B and charcoal. Kits must differ by lamp or sketchbook brand. List specs, brand, review counts, and price.
- Find the fastest-delivery premium pet care bundle under $250: (1) hands-free dog leash with reflective stitching, padded waist belt, and shock absorber; (2) collapsible BPA-free silicone travel bowl; (3) no-pull harness with five-point control and breathable mesh. All rated 4.5+ stars with two-day delivery. List materials, feature highlights, shipping ETA, review counts, and combined price.
- Recommend 2 cordless stick vacuums under $200 with detachable batteries, HEPA filtration, at least 25 minutes runtime, and listed runtime and battery replacement ease.
- Find 3 premium gaming chairs under $300 with adjustable lumbar support, at least 150-degree recline, verified user weight support of at least 300 lbs, and listed weight capacity and user review rating.
- List 2 portable espresso makers under $100 compatible with Nespresso capsules, at least 18 bar pressure, BPA-free certification, and clearly state compatibility, pressure, and BPA-free status. If criteria aren't met, explain the closest match.

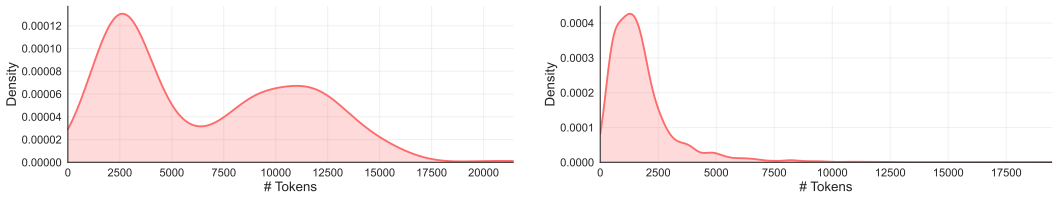
- Recommend 3 noise-cancelling earbuds under $120 with at least 24-hour total playtime, IPX5 or higher waterproof rating, transparency mode, and clearly state sound quality ratings and battery life duration.
- Find 2 smart thermostats under $150 compatible with Alexa and Google Assistant, supporting multi-zone control, energy-saving certification, and clearly state assistant compatibility, multi-zone capability, and certification details. If unavailable, suggest best alternatives with limitations.
- List 3 robot vacuums under $250 with mapping capability, no-go zones, voice assistant support, runtime, and bin capacity. Identify models that fulfill most criteria if any feature is missing.
- Recommend 2 air fryers under $100 with at least five-quart capacity, dishwasher-safe basket, preset cooking functions, and listed cooking presets and ease of cleaning. Highlight trade-offs between price, capacity, and features if necessary.
- Find 3 fitness trackers under $80 with continuous SpO2 monitoring, swim-proof rating of at least 50 meters, sleep-tracking, battery life, and additional health monitoring features clearly listed.
- List 2 electric toothbrushes under $60 with pressure sensors, at least 30-day battery life, ADA acceptance, charging type, and brush head replacement availability. Recommend based on user reviews and clinical backing if ADA acceptance is missing.
- Recommend 3 webcam models under $70 with autofocus, at least 1080p resolution, 60 frames per second streaming, built-in privacy cover, and microphone quality. Clearly identify compromises on frame rate or privacy cover if necessary.
- Recommend 2 wireless routers under $200 with Wi-Fi 6 support, dual-band, gigabit Ethernet ports, and listed max coverage area. If no routers meet all, choose closest and note missing attribute.
- Find 3 kitchen stand mixers under $300 with at least 10 speeds, 5-quart bowl, and metal construction. List speed count, bowl size, and material. If fewer than three, include models that miss one requirement and indicate which.
- List 2 Bluetooth speakers under $150 with waterproof rating of IPX7, at least 12-hour battery life, and built-in voice assistant support. State battery life and assistant type. If criteria aren't met, explain closest fit.
- Recommend 3 DSLR-style mirrorless cameras under $700 with interchangeable lenses, 4K video, and in-body image stabilization. Clearly state sensor resolution, video spec, and stabilization type. If none, suggest best trade-offs.
- Find 2 cordless electric lawn mowers under $400 with at least 45 minute runtime, 20-inch deck, and mulching capability. List runtime, deck size, and whether mulching kit included.
- List 3 external SSDs under $150 with USB-C connection, at least 1 TB capacity, and read speed over 1000 MB/s. Provide capacity, interface, and read speed. If fewer than three, note closest specs.
- Recommend 2 smartwatches under $200 with built-in GPS, NFC payments, and ECG or heart-rate variability tracking. State GPS, NFC, and health feature. If none have ECG, list HRV instead.
- Find 3 midsize camping tents under $200 with capacity for at least four people, full rainfly, and tent weight under 15 pounds. List capacity, rainfly type, and weight. If criteria not fully met, show closest option.
- List 2 home security cameras under $100 with 1080p resolution, night vision, two-way audio, and local storage option. Provide each feature's status. If no local storage, note cloud-only limitation.
- Recommend 3 pair of running shoes under $120 with carbon-fiber plate or equivalent propulsion tech, neutral support, and weight under 10 ounces. State plate tech, support type, and weight. If none, note closest feature set.
- Recommend 2 DSLR lenses under $500 for wildlife photography that have at least 300mm focal length, image stabilization, and autofocus under 0.5s. List focal length, stabilization

type, and measured autofocus time. If none, suggest closest alternatives and note compromises.

- Find 3 camping stoves under $150 that support propane and butane, boil one liter of water in under four minutes, and have built-in wind protection. Provide fuel type compatibility, boil time, and wind guard design details. If fewer than three meet all, include near matches with missing features.
- List 2 insulated tumblers under $40 with 30-hour cold retention, 12-hour hot retention, and dishwasher-safe lid. State retention times, lid type, and size. If criteria are missing, recommend closest and explain trade-offs.
- Recommend 3 external monitors under $300 with at least 27-inch size, IPS panel, 75 Hz refresh rate, and USB-C power delivery. Provide screen size, panel type, refresh rate, and wattage delivered via USB-C. If power delivery is absent, note fallback features.
- Find 2 portable power stations under $400 with AC outlets, solar charging support, and at least 500 Wh capacity. List AC output wattage, solar input type, and capacity. If solar charging not supported, mention alternative recharge methods.
- List 3 noise-monitoring baby monitors under $200 with temperature display, lullaby/music playback, and two-way talk. Provide screen size (or app), temperature reporting, and audio features. If lullaby feature is missing, note which are closest.
- Recommend 2 countertop ice makers under $250 that produce at least 26 lbs of ice per day, have self-clean function, and use bullet-shaped ice. List daily output, cleaning cycle, and ice type. If none match exactly, propose closest and trade-offs.
- Find 3 inflatable paddle boards under $700 with maximum load of at least 300 lbs, included pump, and thickness of at least 6 inches. Provide max load, pump type, and board thickness. If load capacity slightly lower, note it.
- List 2 smart jump ropes under $100 with integrated fitness tracking, Bluetooth app syncing, and rechargeable battery. State tracking metrics, app availability, and battery runtime. If rechargeable is not available, recommend suitable near match.
- Recommend 3 gaming keyboards under $150 with per-key RGB lighting, hot-swappable switches, and dedicated macro keys. Provide switch type, lighting software, and macro implementation details. If hot-swap is not present, note it.
- Recommend 3 sulfate-free shampoos under $25 that are color-safe, contain at least 2% argan oil, and have a pH between 5 and 6. List ingredient percentages, color-safe claims, and pH value. If pH is not listed, note this clearly.
- Find 2 paraben-free retinol serums under $50 with at least 0.5% retinol, added vitamin C, and cruelty-free certification. Clearly state retinol and vitamin C concentrations, and certification status.
- List 3 mineral sunscreens under $30 that are zinc-oxide based, at least SPF 30, reef-safe (no oxybenzone/octinoxate), and water-resistant for at least 80 minutes. Provide SPF, zinc-oxide percentage, and water resistance time.
- Recommend 2 fragrance-free facial moisturizers under $40 with hyaluronic acid, non-comedogenic, and dermatologist-tested. List hyaluronic acid percentage, comedogenic rating, and dermatologist testing claims.
- Find 3 sulfate-free cleansing oils under $35 with at least two plant oils, vitamin E, and eco-cert organic certification. Clearly state oil types, vitamin E content, and certification status.
- List 2 paraben-free body lotions under $20 with at least 10% shea butter, fast-absorbing formula, and allergy-tested status. Provide shea butter percentage, absorption claim, and testing certification.
- Recommend 3 vegan lipsticks under $25 that are cruelty-free, provide at least six-hour wear, and include SPF 15. List wear time, SPF rating, and certification details. If SPF is absent, clearly state the trade-off.
- Find 2 alcohol-free facial toners under $30 with probiotics or niacinamide, non-pore-clogging, and a pH between 4 and 5. List active ingredients and pH; if pH is not listed, clearly note it.

- List 3 caffeine-infused eye creams under $35 that are fragrance-free, paraben-free, and claim to reduce puffiness within 15 minutes. State caffeine concentration, effectiveness claim timing, and ingredient exclusions.
- Recommend 2 multifunctional balm sticks under $30 that are petroleum-free, contain SPF, and can be used on lips, cheeks, and cuticles. Provide ingredient list, SPF rating, and specified usage areas. Clearly note if multifunction use is limited.

C Experiment Results

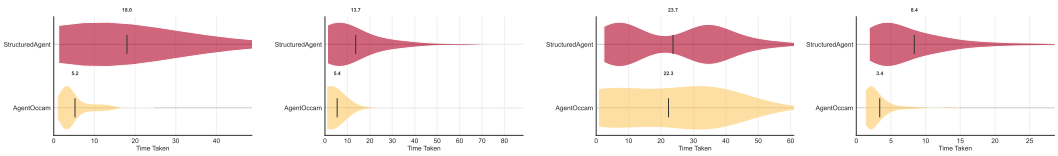


(a) WebVoyager (b) WebArena (Shopping)

Figure 6: Distribution of average HTML observation lengths (in tokens) per timestep across agent trajectories on WebVoyager and the WebArena Shopping category. Token counts reflect only raw web page content, excluding any additional context provided to the agent.

Method	**Map** (81)	**Shop.** (126)	**Reddit** (11)	**GitLab** (63)	**Overall** (281)
StructuredAgent	0.348	**0.432**	0.545	0.383	**0.401**
AgentOccam	**0.410**	0.370	**0.591**	**0.410**	0.399

Table 6: Success rates on WebArena using Kimi-K2 as the backbone model, evaluated on the subset of tasks with URL and string evaluators.



(a) GITLAB (b) MAP (c) REDDIT (d) SHOPPING

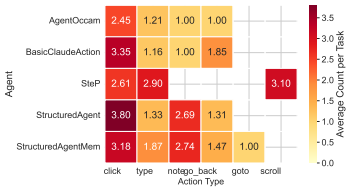
Figure 7: Completion time distributions (in minutes) for StructuredAgent and AgentOccam across four WebArena task categories, with the mean marked per violin. StructuredAgent exhibits longer completion times and heavier right tails, reflecting greater variance in task duration due to its deliberate planning and dynamic replanning upon encountering failures.

D Detailed Ablation Analysis

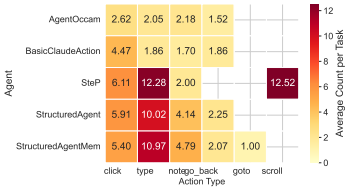
Tables 7 and 8 report And/Or tree statistics for all four ablation conditions on the Amazon-easy and Amazon-hard splits.

Metric Definitions.

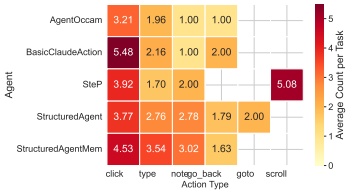
- **Success Rate** is the task success rate as judged by the specified LLM evaluator (Gemini or Opus).
- **Constraint Satisfaction** is the fraction of user-specified constraints satisfied across all tasks, including partial successes.
- **Browser Steps** is the mean number of browser actions executed per task.
- **Max Depth** is the mean maximum depth of the And/Or tree.



(a) Amazon-easy

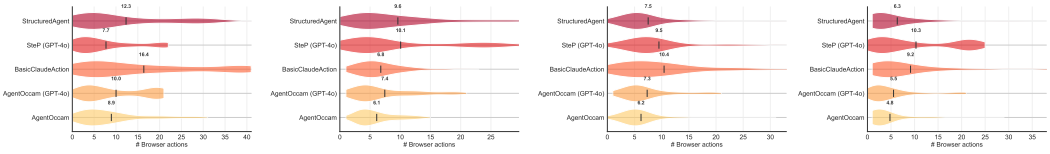


(b) Amazon-hard



(c) WebVoyager Other

Figure 8: Average action counts per task for each agent on Amazon and WebVoyager Other tasks, broken down by action type (click, type, note, go_back). STRUCTUREDAGENT and StructuredAgentMem issue more `type` and `note` actions on harder tasks because they gather and record more information about candidate items.



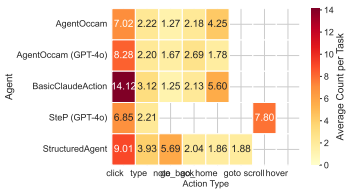
(a) GitLab (b) Map (c) Reddit (d) Shopping

Figure 9: STRUCTUREDAGENT's longer trajectories on harder tasks (Map, GitLab) reflect systematic exploration of alternative strategies, which translates to higher overall success rates (Table 1). On Shopping and Reddit, where STRUCTUREDAGENT achieves the largest gains, trajectory lengths remain comparable to AgentOccam.

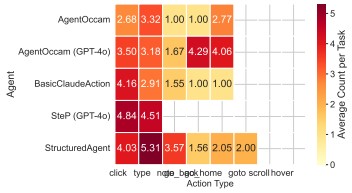
- **Total, AND, OR, and Action Nodes** are mean node counts by type.
- **Pruned Nodes** is the mean number of nodes marked as pruned because they are irrelevant or irrecoverable.
- **Tree Efficiency** is the fraction of nodes that contributed to the final solution, meaning nodes that were neither pruned nor deleted.
- **Repair Trigger Rate** is the fraction of tasks with at least one node failure that invokes repair when repair is enabled.
- **Recovery Rate** is the fraction of repair attempts that restored the failed node to a passing state.
- **Plan Preservation** is the fraction of originally planned nodes that remained in the final tree without deletion.
- **Total Revisions** is the mean total number of tree modification operations, counting expansions, repairs, and prunings.
- **Nodes Revised** is the mean number of distinct nodes modified by at least one revision.

E Error Localization Experiment

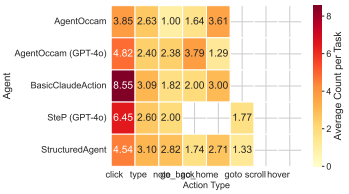
We test whether the hierarchical structure of STRUCTUREDAGENT makes its failures easier to diagnose than those of AgentOccam. We run the experiment on the 15 Amazon tasks that both agents failed, where the STRUCTUREDAGENT root node is marked as pruned and the AgentOccam task remains incomplete. The design controls for prompt phrasing, judge identity, and forced choice scoring.



(a) GITLAB



(b) MAP



(c) REDDIT

Figure 10: Average action counts per task for each agent across three WebArena task categories (GitLab, Map, Reddit), broken down by action type (click, type, note, go_back, go_home). STRUCTUREDAGENT issues more `note` actions because it records structured task information. Its click and type counts are similar to the baselines.

Table 7: AND/OR tree analysis on the Amazon-easy split with a Haiku 4.5 backbone. Full uses both NODE REPAIR and OR alternatives, OR only disables repair, Repair only disables OR nodes, and Neither disables both. All rates are percentages and count-valued metrics report means. Repair Trigger Rate counts node failures that would invoke repair, whether or not repair is enabled. The no-repair conditions make no repair attempts, so their Recovery Rate is reported as 0 by convention.

	Full	**OR only**	**Repair only**	**Neither**
Task Performance				
Success Rate (Gemini)	68.3	65.0	75.0	60.0
Success Rate (Opus)	67.5	50.0	57.5	50.0
Constraint Satisfaction	89.3	83.5	87.7	77.2
Tree Structure				
Browser Steps	19	8	22	8
Max Depth	3.6	3.5	4.1	3.1
Total Nodes	34	13	34	11
AND Nodes	3.4	3.0	4.0	2.4
OR Nodes	0.2	0.1	0.0	0.1
Action Nodes	18	7	21	7
Repair & Pruning				
Pruned Nodes	12.2	3.0	9.2	2.2
Tree Efficiency	74	73	78	76
Repair Trigger Rate	22	32	25	18
Recovery Rate	44	0	70	0
Plan Preservation	99	93	98	96
Tree Revisions				
Total Revisions	64	24	68	20
Nodes Revised	33	13	34	11

Diagnosis. We pass the full trajectory of each agent to a fixed diagnosis model, Claude Opus 4.6, which returns a structured diagnosis. The diagnosis states the root cause, the critical steps where the agent went wrong, a recommended fix, a failure category, and a confidence score. Each agent is represented by the trajectory format it natively produces. AgentOccam receives its flat sequence of steps together with the interaction history notes it records at each step. STRUCTUREDAGENT receives its AND/OR decomposition tree, in which every node carries an execution status of `success`, `fail`, `pruned`, or `visited`, followed by the same sequence of steps and notes. We require every

Table 8: AND/OR tree analysis on the Amazon-hard split with a Haiku 4.5 backbone. Conditions and conventions are as in Table 7. All rates are percentages and count-valued metrics report means.

	Full	**OR only**	**Repair only**	**Neither**
Task Performance				
Success Rate (Gemini)	38.3	13.3	35.0	16.7
Success Rate (Opus)	14.5	8.5	20.0	6.8
Constraint Satisfaction	82.8	59.8	77.8	57.3
Tree Structure				
Browser Steps	43	11	46	12
Max Depth	5.3	3.6	5.6	3.8
Total Nodes	80	20	84	20
AND Nodes	6.1	3.2	7.0	3.4
OR Nodes	0.6	0.1	0.0	0.1
Action Nodes	42	10	45	11
Repair & Pruning				
Pruned Nodes	29.3	6.2	32.1	5.0
Tree Efficiency	62	59	64	65
Repair Trigger Rate	63	58	70	60
Recovery Rate	37	0	43	0
Plan Preservation	95	88	96	88
Tree Revisions				
Total Revisions	172	36	190	36
Nodes Revised	79	20	83	20

claim in a diagnosis to rest on the observations, actions, and notes in the trajectory, and we pass the trajectory in full without truncation.

Blinded comparison. We present the two diagnoses to a judge as "Agent A" and "Agent B" and randomize their order for each task under a fixed seed, so the judge never learns which architecture produced which diagnosis. The judge rates four dimensions. Precision measures how exactly the diagnosis locates the failure. Self sufficiency measures whether the diagnosis lets an engineer repair the agent without rereading the trajectory. Clarity measures how easily a reader understands the failure. Actionability measures how directly the recommended fix can be implemented. The judge also states an overall preference. On every item the judge chooses A, B, or Tie. We map the verdicts back to STRUCTUREDAGENT and AgentOccam only after collecting them. We use two judges that differ from the diagnosis model, Gemini 2.5 Flash and Claude Opus 4.8.

Prompt variants. To rule out sensitivity to a single phrasing, we run the diagnosis stage under four paraphrases (detailed, terse, chain of thought, and free category). We run the comparison stage under four paraphrases that cross two question wordings with two tie tolerance framings. Each judge therefore produces $15 \times 4 \times 4$ verdicts for every dimension.

Results. STRUCTUREDAGENT wins on every dimension for both judges (Table 9). Pooled across judges and paraphrases, the judges prefer the STRUCTUREDAGENT diagnosis on 78.5% of the decided overall preference verdicts at $p < 10^{-36}$. The largest margin is on Precision, where STRUCTUREDAGENT wins 90.1% of decided verdicts pooled across judges. This is consistent with the mechanism we expect, since the tree with per-node execution statuses points the diagnosis model directly at the sub-goal that failed, whereas a flat trajectory forces it to reconstruct the failure point from the step sequence alone.

Table 9: Blinded comparison of STRUCTUREDAGENT and AgentOccam diagnoses across 15 tasks that both agents failed, 4 diagnosis paraphrases, and 4 comparison paraphrases. Each cell reports the percentage of decided verdicts that favor STRUCTUREDAGENT, followed by the raw counts SA/AO/Tie. The diagnosis model, Claude Opus 4.6, differs from both judges. We randomize the presentation order for each task under a fixed seed. We test significance with a one sided binomial test that compares STRUCTUREDAGENT against AgentOccam and excludes ties. The markers *, **, and *** denote $p < 0.05$, $p < 0.01$, and $p < 0.001$.

Dimension	Gemini 2.5 Flash	Opus 4.8	Pooled
Precision	86.8 (203/31/3)***	94.7 (160/9/70)***	90.1 (363/40/73)***
Self sufficiency	64.3 (9/5/223)	72.4 (155/59/25)***	71.9 (164/64/248)***
Clarity	63.2 (91/53/93)***	76.9 (183/55/1)***	71.7 (274/108/94)***
Actionability	78.3 (162/45/30)***	67.6 (148/71/20)***	72.8 (310/116/50)***
Four dimension pooled	77.6 (465/134/349)***	76.9 (646/194/116)***	77.2 (1111/328/465)***
Overall preference	81.7 (192/43/2)***	75.2 (170/56/13)***	78.5 (362/99/15)***

You are an expert evaluator analyzing web browsing agent trajectories to diagnose why a task failed.
You will be given: the task objective, which constraints were NOT satisfied, and the agent's step-by-step trajectory.
Your job is to perform a precise root-cause analysis of why the agent failed.
You MUST respond in this exact JSON format:
{ "root_cause": "<one sentence: the primary reason the task failed>", "critical_steps": [{"step_number": <int>, "what_happened": "<what the agent did>", "what_went_wrong": "<why this was the critical error>"}], "what_should_have_been_done": "<specific alternative action(s)>", "failure_category": "<one of: wrong_search_strategy | missed_information | navigation_error | premature_stop | constraint_violation | verification_failure | other>", "confidence": <1-5 integer> }
EXAMPLE of a valid response (structure only – your content will differ):
{ "root_cause": "The agent searched with the wrong keyword and never applied the price filter.", "critical_steps": [{"step_number": 2, "what_happened": "Typed 'cheap laptop' into search", "what_went_wrong": "Ignored the required 'under $500' constraint, returning irrelevant results"}, {"step_number": 5, "what_happened": "Clicked the first result and stopped", "what_went_wrong": "Never verified the price met the constraint"}], "what_should_have_been_done": "Search 'laptop', then apply the <$500 price filter and sort by rating before selecting.", "failure_category": "wrong_search_strategy", "confidence": 4 }
Output ONLY the JSON object – no prose before or after, no markdown code fences. Use exactly these keys, once each.
GROUNDING (mandatory): Every claim in your diagnosis MUST be grounded in or directly implied by the observations, actions, and notes in the trajectory. Do NOT assume, infer, or invent any fact that is not present in the trajectory. If the trajectory does not make the root cause clear, say so explicitly in "root_cause" and set a LOW "confidence" (1-2) rather than guessing. Do not use hedging like "likely" or "probably" to smuggle in unsupported claims.
Guidelines: - Be SPECIFIC. Name the wrong search term, the missed filter, the unexplored page. - Point to exact step numbers where the critical error occurred. - The fix should be concrete and actionable. - Confidence 5 = root cause obvious; 1 = unclear.

Table 10: Diagnosis prompt (detailed variant) that we use in the error localization experiment. We append the agent trajectory below this system prompt. AgentOccam receives its flat step sequence with the notes it records at each step. STRUCTUREDAGENT receives its AND/OR tree with per-node status followed by its step sequence with the notes it records at each step.

F Human Evaluation

To validate the reliability of our LLM-based evaluation, we conducted a human evaluation study on the Amazon-hard task set. We recruited one human annotator who was not an author of this paper. We provided the annotator with the description of the AND/OR tree structure used by each agent and the full agent trajectories, including all browser observations and actions taken. For each of the 60 Amazon-hard tasks, the annotator judged whether each method fully completed the task, and we report the percentage of tasks that each method completed.

Table 11: Human evaluation task-success rate (%) on the 60 Amazon-hard tasks. A single independent annotator (non-author) judged whether each method fully completed each task from its full trajectory. Table 13 shows an example where the annotator overlooked unmet constraints and marked a partially correct answer as complete.

Method	**Human**
STRUCTUREDAGENT	**56.7**
AgentOccam	55.0
BasicClaudeAction	40.0

The human evaluation scores (Table 11) agree with the LLM judge averages in Table 2. STRUCTUREDAGENT achieves the highest human score, followed by AgentOccam, and BasicClaudeAction trails both, which is the same ordering the automated judges produce.

Table 12: Evaluator reliability on the 60 Amazon-hard tasks, per judge and agent. Agr. and Cohen's κ measure how well each judge's per-task success labels match the human annotator's. Sat. is the judge's mean constraint-satisfaction rate, the mean of $(\text{satisfied}+1)/(\text{total}+1)$ across tasks.

Judge	Agent	Agr. (%)	κ	Sat. (%)
GPT-4.1-mini	StructuredAgent	81.7	0.64	87.2
	AgentOccam	75.0	0.50	90.8
	BasicClaudeAction	86.4	0.71	82.7
Gemini-2.5-Flash	StructuredAgent	71.7	0.45	82.1
	AgentOccam	68.3	0.38	84.8
	BasicClaudeAction	86.7	0.71	78.9
Kimi-K2	StructuredAgent	76.7	0.54	81.5
	AgentOccam	75.0	0.50	87.8
	BasicClaudeAction	86.7	0.71	80.5

Table 13: Case study of task Amazon–DeepMulti-603. The task asks the agent to recommend three wireless printers under $150 that support both AirPrint and borderless 4×6 photo printing, to list brand, print support, and price, and not to add to cart. The printer rows reproduce each agent's returned answer. ✓ and ✗ mark whether each requested spec is grounded in the page observations rather than only asserted by the agent. AgentOccam asserts that all three of its printers meet the specs, but the observations confirm this for only the TS5320. StructuredAgent states in its answer that none of its printers fully meet all requirements. The human annotator nevertheless marks AgentOccam as a success, while the LLM judge, which requires every spec to be grounded in the observations, marks both agents as failures.

Agent	Recommended printer (price)	Wireless, <$150	AirPrint	Borderless 4×6
AgentOccam	Canon TS5320 ($85.99)	✓	✓	✓
	Epson XP-5200 ($79.99)	✓	✗ *(not observed)*	✓
	Canon TS702a ($109.00)	✓	✓	✗ *(not observed)*
StructuredAgent	Canon PIXMA TS6420a ($44.99–109.95)	✓	✓	✗ *(not observed)*
	Canon PIXMA TR4720 ($64.00)	✓	✗ *(not observed)*	✓
	HP DeskJet 2827 ($\cdots$)	✓	✗	✗
	From the agent's answer: "none fully meet all requirements."			
Judge score		AgentOccam: 0.75	StructuredAgent: 0.50	
Human verdict		AgentOccam: **Yes**	StructuredAgent: **No**	
LLM judge verdict		AgentOccam: **Failure**	StructuredAgent: **Failure**	

Table 12 quantifies how closely each LLM judge aligns with the human annotator on the 60 Amazon-hard tasks, per agent.

G Prompts

H Limitations

StructuredAgent incurs higher computational overhead than flat agents due to additional LLM calls for node expansion, repair, and verification. The quality of the hierarchical plan depends on the backbone LLM's ability to produce meaningful decompositions; poor expansions can cascade into subtree failures. Our framework currently operates on text and HTML observations only, without visual understanding of rendered web pages.

I Ethical Considerations

This work presents a planning framework for web-browsing agents evaluated on publicly available benchmarks and commercial websites. We do not collect, store, or process personal user data. All experiments are conducted in sandboxed or read-only browsing environments and do not involve human subjects. The human annotator participated voluntarily and was compensated fairly for their

You are a web-browsing assistant designed to extract structured constraint data from user queries. Given a natural language query, your task is to identify and return:
1. A list of **distinct items** the user wants to retrieve (if any). 2. For each item, a list of **item-level constraints** that are explicitly mentioned in the query. 3. A separate list of **task-level constraints** that apply to the overall task.
Definitions:
- **Item**: A specific product, object, or entity that the user wants to find or retrieve information about. - **Item-level constraint**: Any attribute or specification that is directly tied to an individual item. This may include (but is not limited to): brand, model, price, rating, features, color, size, dimensions, delivery time, material, and any other **explicitly** stated property of the item. - **Task-level constraint**: A condition that affects the overall search or task execution rather than a specific item.
Instructions:
- You must extract **only those task-level and item-level constraints that are explicitly stated** in the query. - You must **not assume, infer, or interpret** any implicit constraints, even if they seem logically implied or contextually likely. - Do **not include any reasoning, commentary, or explanation** in your output. - Each item type must appear only once in the 'ITEM_CONSTRAINTS' list. - If the query contains no valid constraints or items, return empty lists using the required format.
Output Format:
The result must be returned strictly in the following JSON format:
{ "TASK_CONSTRAINTS": ["explicit_task_constraint1", "explicit_task_constraint2", "..."], "ITEM_CONSTRAINTS": [{ "item_type": ["explicit_item_constraint1", "explicit_item_constraint2", "..."] }, ...] }
EXAMPLE:
QUERY: Find a black laptop bag under $40 and a wireless mouse under 25. Both should be delivered within 2 days. URL: www.amazon.com
OUTPUT:
{ "TASK_CONSTRAINTS": [ ], "ITEM_CONSTRAINTS": [{ "laptop bag": ["Color: black", "Price: under $40", "Delivery time: within 2 days"] }, { "wireless mouse": ["Price: under $25", "Type: wireless", "Delivery time: within 2 days"] }] }
QUERY: {task_objective}

Table 14: Prompt for extracting task constraints for task completion evaluation.

time. Autonomous web agents can take real actions on live websites, such as submitting forms or completing purchases. Deployments outside sandboxed settings should therefore restrict the action space and require human confirmation for irreversible steps.

J Future Work

Several directions remain open. LLM-based scoring of OR node alternatives could be replaced with learned value functions. Integrating vision-language models would extend the framework to visually rich interfaces. The framework could also be applied to other domains such as code generation and OS-level tasks. Finally, caching and policy distillation could reduce computational cost.

K POMDP Formalization

We model the agent-browser interaction as a Partially Observable Markov Decision Process (POMDP) $\langle \mathcal{O}, \mathcal{S}, \mathcal{A}, R, T, p_0, \gamma \rangle$ Kaelbling et al. [1998], where $o \in \mathcal{O}$ is an *observation* comprising the task description, agent-constructed plans, the HTML DOM, and interaction metadata; $s \in \mathcal{S}$ is the true underlying *state* of the browser and agent; and $a \in \mathcal{A}$ is an *action*, either an atomic browser operation (click, scroll, type, go back) or a token sequence describing a sub-goal or strategy. The *reward function* $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ assigns a scalar reward to each state-action pair, the *transition function* $T : \mathcal{S} \times \mathcal{A} \to \Delta^{|\mathcal{S}|}$ defines a distribution over next states, $p_0 \in \Delta^{|\mathcal{S}|}$ is the *initial state distribution*, and $\gamma \in (0, 1)$ is the discount factor. The agent's *policy* $\pi : \mathcal{S} \to \Delta^{|\mathcal{A}|}$ maps states to distributions over actions and is, in general, history-dependent [Kaelbling et al., 1998]. We represent the agent's policy using an LLM. At each timestep t, the observation history $h_t = \{o_1, \ldots, o_t\}$ is encoded as a token sequence $x_t = [x_1, \ldots, x_m]$ serving as the input prompt. The action $a_t = [y_1, \ldots, y_n]$ is then generated autoregressively as:

$$\Pr(a_t \mid h_t) = \prod_{i=1}^{n} \pi(y_i \mid x_1, \ldots, x_m, y_1, \ldots, y_{i-1}).$$

L Detailed DFS Example Trace

We provide a step-by-step trace of the modified greedy DFS on the task "`Find a vegan chocolate brownie recipe with 4+ rating on allrecipes.com`" (illustrated in Figure 2). The agent begins with only root node 1 on the stack in ENTERING. After each ACTION execution, OBSERVATION SUMMARIZER updates the internal context and GLOBAL TREE UPDATE is invoked (making no tree modifications in this trace).

Steps 1–2 [Expand root & execute 1.1]: The agent pops node 1 and invokes NODE EXPANSION, which identifies it as an AND node with three ordered children: 1.1 (type query), 1.2 (select recipe),

You are an evaluator assessing the performance of a web-browsing agent on a specific task.
You will be provided with the following components:
- Task Description A natural-language instruction outlining the goal the agent is expected to accomplish. This may involve retrieving, summarizing, comparing, or verifying information from the web.
- Constraints A list of specific item-level and task-level requirements that must be satisfied for the task to be considered successful.
- Trajectory Summary A step-by-step log of the agent's interaction with the environment. Each time step includes: - Observation: Summary of the web-page the agent perceives at that moment. - Action: The behavior or command the agent executes (e.g., clicking, searching). - Interaction History Summary: Summary of the agent's progress (if available)
- The agent's final response to the task
- Evidence extracted from raw observations A digest of what was actually observed across the agent's full trajectory.
Evaluation Guidelines
- **Evaluate each item-level constraint once per item type**. Do not repeat the constraint evaluation for each instance of the same item type. Instead, assess whether the constraint is satisfied **across all relevant items of that type**, and justify the evaluation with evidence from the observations, notes, or interaction history. **For example:** If the task requires finding 3 laptops under $1000, and the agent recommends 3 laptops, evaluate the constraint "Price : under 1000" only once. Mark it as **satisfied only if all 3 laptops** meet the price constraint.
- Only use information that is **explicitly visible** in the agent's observations, actions, notes, and final response. Do not rely on external knowledge, domain expertise, or assumptions made by the agent.
- Do **not assume that a constraint is satisfied** based on the agent's search terms, reasoning, or keyword usage (e.g., searching for "size 6"). A search query does **not guarantee** that the results meet the constraint. A constraint is only considered satisfied if the specific value (e.g., "Size: 6") is explicitly shown in the observations, actions or agent's notes.
- If the task requires finding the cheapest, most rated, or most reviewed item, only verify that appropriate filters or sorting options were applied - but **only if those options are actually available on the website**.
- Evaluate only based on the **final response** of the agent. Intermediate steps or partially explored options should not be considered unless they directly contribute to the final result.
- Do **not award partial credit** for incomplete constraints. A constraint is either **fully satisfied** or **not satisfied**. All requested attribute values (e.g., price, brand, size) are treated as **separate constraints**.
- Do **not evaluate** the agent based on constraints that are **not explicitly stated** in the task description. For example, if the task does not mention "size," "material," or "capacity," do not treat them as constraints — even if the agent includes or discusses them during the task.
- If there is no final response or the task is incomplete, assign a score of 0.
- You do not need to consider constraints that require the agent to find items with a given delivery period as the agent does not have access to the current date and time.
Scoring Instructions
- Assign **1 point** for each item-level or task-level constraint that is fully and explicitly grounded in the observations, notes and actions. **Confirm that the agent is not hallucinating.** **Note:** If the recommended item costs **less than** the specified price, it still satisfies the price constraint. Evidence for constraints satisfied must be grounded only in observations, notes and actions. **Do not** ground evidence in the final response of the agent.
- Assign **0 points** for constraints that are: - Not satisfied or Partially Satisfied - Assumed or inferred without explicit evidence in observations, notes, or final response
- Assign **1 point** if the task was completed successfully and the final response is likely **not hallucinated**. A task is considered **completed successfully** if the agent provides a clear and specific final response that directly fulfills the task objective, and is **explicitly supported** by observed content or notes.
- Normalize the score: **Score = (Number of Satisfied Constraints + 1 point if task completed) / (Total Number of Constraints + 1)** This yields a final score between **0.0 and 1.0**.
Final Verdict
- **SUCCESS** – Only if **all** explicitly stated item and task-level constraints are fully satisfied (score = 1.0) - **FAILURE** – If **any** constraint is not satisfied, or if the task is incomplete (score < 1.0)
Your response must strictly follow this format:
1. **Task Overview** - **Task Description**: (Copy the task as provided.)
- **Constraints**: (List the item-level and task-level constraints exactly as stated in the task description. **Do not infer or add any additional constraints.**)
2. **Evaluation of Each Constraint** Evaluate each constraint using one of the following labels: - Satisfied - Partially Satisfied - Not Satisfied
Constraint 1: Evidence: Evaluation: Reason:
Constraint 2: Evidence: Evaluation: Reason:
(Continue for all constraints.) **Note**: Evidence must be grounded in observations, notes and actions only. Evidence cannot be grounded in the final response of the agent.
3. **Agent Behavior Assessment** - **Action Effectiveness**: Was the agent's sequence of actions appropriate and goal-directed?
- **Interpretation Accuracy**: Did the agent correctly interpret the content it observed?
- **Handling of Multi-step Tasks (if applicable)**: Did the agent address all required components of the task?
4. **Final Scoring** - Total Satisfied Constraints: X - Total Constraints: Y - Score (X / Y): Z - Final Verdict: SUCCESS / FAILURE
5. **Summary Comments (Optional)** (Provide any additional insights or suggestions about the agent's behavior or reasoning.)
OBJECTIVE: {objective}
CONSTRAINTS: {constraints}
FINAL RESPONSE: {notes}
TRAJECTORY {trajectory}

Table 15: Prompt for evaluating completion of task based on observation summaries, actions, notes and final response of the agent.

You are a meticulous evidence-EXTRACTION assistant. You are NOT the judge. You are shown the raw text of ONE web page OBSERVATION (an accessibility tree / page dump) captured during a browsing session, plus the OBJECTIVE and CONSTRAINTS. Your only job is to COPY OUT, verbatim, every snippet in THIS observation that bears on any constraint or on overall task completion, and return it as JSON.
VERBATIM RULE (most important): every "quote" you emit MUST be an EXACT, character-for-character substring of THIS observation's raw text – copy-pasted. Do NOT reword, summarize, normalize, round, translate, or reformat. If a price appears as "-31 109.98 List:159.99", copy exactly that; never rewrite it as "$109.98". If a title spans a long string, copy the whole literal string. A reader must be able to Ctrl-F your quote and find it unchanged in this observation.
BANNED – never report content in your own words ("the page shows", "it lists", "priced at", etc.). Every "quote" is a raw verbatim substring; nothing else.
OUTPUT FORMAT – return ONLY a JSON object, no prose, no markdown fences: {"items": [{"constraint": "<the constraint label from CONSTRAINTS this snippet bears on, verbatim>", "quote": "<exact verbatim substring copied from THIS observation>", "step_id": <the integer step id given below>}]} - One entry per distinct relevant snippet. If several constraints share one snippet, emit one entry per constraint. - If NOTHING in this observation bears on the task, return exactly: {"items": []}
Rules: - Copy real text from THIS observation only. Never invent. Never infer. Never reference other steps. - Do NOT read or rely on any agent's summary or answer – extract only from the raw observation text. - Do NOT judge. No verdict/status/agreement words. Only the JSON object.
EXAMPLE (illustrative only – a made-up museum page, unrelated to your actual task): CONSTRAINTS: ["Ticket price: under 20 EUR", "Open on Mondays"] THIS OBSERVATION (step_id=2): "link 'Rijks Gallery' StaticText 'Adult admission EUR 18,50' StaticText 'Closed on Mondays' button 'Buy'" Correct output: {"items": [{"constraint": "Ticket price: under 20 EUR", "quote": "Adult admission EUR 18,50", "step_id": 2}, {"constraint": "Open on Mondays", "quote": "Closed on Mondays", "step_id": 2}]} Note the quotes are copied character-for-character (e.g. "EUR 18,50", not "18.50 euros").

Table 16: Prompt for extracting grounded, verbatim evidence from each raw observation. The per-observation snippets are consolidated into the evidence digest supplied to the evaluator.

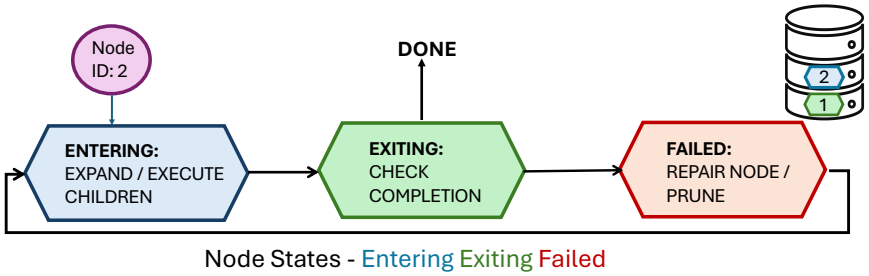


Figure 11: Illustration of node state transitions during iterative modified greedy depth-first search on AND-OR tree. Unlike traditional depth-first search, nodes may be revisited and repeatedly transition through ENTERING, EXITING, and FAILED states.

and 1.3 (note recipe), all of which must succeed. Node 1 is re-pushed in EXITING and its children are pushed in ENTERING in prescribed order. Node 1.1 is then popped, identified as ACTION, executed, and marked SUCCESS.

Steps 3–4 [Expand 1.2 **& execute** 1.2.1**]:** The agent pops node 1.2 and identifies it as an OR node encoding alternative recipe-selection strategies, with children 1.2.1 (Recipe A, score 1.0) and 1.2.2 (Recipe B, score 0.95). Only the highest-ranked child 1.2.1 is pushed. It is popped, executed as ACTION (clicking Recipe A), and NODE COMPLETION CHECK confirms the OR objective of 1.2 is satisfied, so 1.2.2 is never explored and 1.2 is marked SUCCESS.

Steps 5–6 [Execute 1.3 **& terminate]:** Node 1.3 is popped, executed as ACTION (noting the recipe), and marked SUCCESS. Root node 1 is then popped in EXITING; NODE COMPLETION CHECK confirms all three AND children have succeeded. The stack empties and the agent terminates with SUCCESS.

No node enters FAILED in this trace, so repair and pruning are not triggered. Had node 1.2 exhausted both OR alternatives without success, it would enter FAILED, causing sibling 1.3 to be pruned and failure to propagate to root node 1, which would then invoke NODE REPAIR if its revision budget permits.

M Tree and Agent Operations Details

M.1 Operator Details

NODE EXPANSION: This operator identifies a node's type and, when appropriate, generates its children. For AND nodes, it generates sub-goal descriptions; for ACTION nodes, it produces a browser-level command; and for OR nodes, it outputs a set of candidate strategies along with their scores.

You are an efficient AND/OR Tree Constructing Agent specialized in web-browsing tasks. You solve complex problems using AND/OR planning trees. You dynamically construct AND/OR planning trees from observations of the webpage's HTML DOM structure for efficient and robust task execution.
You are provided:
- root-level task description
- task constraints at item level
- HTML DOM structure as the observation
- node_id and desription of the node to analyze
- summary of current task progress
- summary of notes taken so far
- Information about node's siblings AND node's parent's siblings.
Node definitions:
Node Types <AND/ OR / Atomic>
- AND node: Represents an ordered list of logical sub-goals required to achieve the node's objective.
- OR node: Represents alternative sub-strategies (which can be other AND/OR nodes)
- Atomic Action: Single executable action strictly matching navigation_specifications described below.
Node status indicators:
- Unvisited nodes marked UNVISITED
- Pruned nodes marked PRUNED
- Completed nodes marked SUCCESS
- Temporarily failed nodes marked FAIL
NAVIGATION_SPECIFICATIONS: <LIST OF BROWSER ACTIONS>
Your Task
For the given node:
1. First, briefly analyze the node type: Based on the available information and the node description, determine whether the node is an AND node, an OR node, or an Atomic node. Justify your reasoning before taking any action.
2. Then, choose ONE of the following options for the node:
A. MARK node as Atomic if: - the goal can be achieved using a single atomic action that strictly matches the NAVIGATION_SPECIFICATION. OR - the goal does not require any browser navigation actions like 'type', 'click', and 'go_back' and is information retrieved from thr webpage that can be noted down using the 'note' action.
B. EXPAND the node if: - the goal is not atomic and requires executing a sequence of atomic actions from NAVIGATION_SPECIFICATIONS. Clearly state the node type (AND/OR): - Do not consider sub-goals and sub-strategies that cannot be executed on the webpage using one action or a sequence of actions from NAVIGATION_SPECIFICATIONS. - For AND nodes, provide ordered list of logical sub-goals that are necessary for satisfying the node's objective. - For OR nodes, provide observable list of alternative sub-strategies only. - For OR nodes, order alternatives by likelihood of success, assigning scores within the range (0,1). - Do not add speculative or redundant sub-goals.
VERY IMPORTANT RULES: - **Focus on expanding nodes in a way that will result in completing the task faster with high probability (Don't take very risky actions)**.
- Make sure that the temporal order of the children of an AND node is correct, detailed, and efficient. - Make sure to not split notes into multiple atomic actions.
- Strictly do not use 'note' for noting future actions or sub-goals. For example, this is not a note -> '[Need to examine each search result for: 1. Publication date 2024 2. Review count 20+ 3. Confirm it's a Japan travel guide book]'
- Use 'go_back' action to return to previous page especially when you have consecutive atomic actions like 'click', 'type', 'goto'.
- Atomic actions like 'type', 'click' must be accompanied with valid IDs from the HTML DOM observation.
- Use atomic action 'note' strictly to note information that is necessary for completing the task or for actions that cannot be executed on the webpage.
- Expand only non-atomic steps. Do not decompose goal when it is directly achievable using an action from NAVIGATION_SPECIFICATIONS.
- Create OR nodes to consider different alternative strategies in cases where failure is probable.
- Structure plans as a knowledgeable efficient human would.
Output Format:
Begin by briefly describing the key elements from the input (overall task description, task progress, task feedback, task notes, node description, observation). Then analyze the node within the given context and clearly identifying its type.
Do not change the node's intended objective or assume a different node objective. Your highest priority is satisfying the node's objective.
You can borrow notes from the notes summary if it helps in achieving the node's objective. Use the local tree information to ensure that you are not repeating any sub-goals that have already been considered.
If the node is an Atomic Action, determine and briefly analyze the 3 best possible actions it could represent. For each, explain why it fits the context. Then, select the single best action among them that is not very risky and will contribute to faster completion of the task and justify your choice.
Conclude your response using ONE of the following formats:
FORMAT 1 (Enclose all values in «» on a single line):
Node ID: «node_id»
Node Description: «Describe what does the atomic action do»
Node Type: «Atomic»
Expansion: «Exact Atomic action from NAVIGATION_SPECIFICATIONS»
Reasoning: «Brief justification explaining why the node is classified as Atomic» OR
FORMAT 2 (Enclose all values in «» on a single line, colon (;) separated):
Node ID: «node_id»
Node Description: «node_description»
Node Type: «AND / OR»
Expansion: «1. First sub-goal; 2. Second sub-goal; 3. Additional sub-goals» OR «1. Alternative strategy; 2. Alternative strategy; Additional alternatives as necessary;...»
Reasoning: «Brief justification explaining why the node is classified as AND, OR, or Atomic, based on current observations.»
Example 1: Node ID: «2.1»
Node Description: «Find recipe with visible ratings and review counts»
Node Type: «Atomic»
Expansion: «click [123]»
Example 2:
Node ID: «2»
Node Description: «Filter results by rating and number of reviews»
Node Type: «AND»
Expansion: «1. click [Recipe with 69 ratings to check star rating]; 2. go_back; 3. click [Check page 2 if first recipe doesn't meet criteria]; 4. go_back; 5. click [Check page 3 if needed]»
Example 3:
Node ID: «0.5.1»
Node Description: «Filter recipes by rating and review count»
Node Type: «OR»
Expansion: «1. Manually scan recipe listings for review counts and ratings (score: 0.9), 2. Click through to individual recipes to check ratings and reviews (score: 0.8)»
Do not output anything outside of the specified format. Atomic actions must strictly match format specified in NAVIGATION_SPECICATIONS.
NOTE: Atomic actions 'click' and 'type' must be accompanied with valid IDs from the HTML DOM observation.
ROOT-LEVEL TASK DESCRIPTION:
{task_description}
ITEM LEVEL CONSTRAINTS:
{item_constraints} {additional_context}
OBSERVATION: {observation}
NODE_ID : Description: node_id : {node_description}
LOCAL TREE INFORMATION: {local_tree_info}

Table 17: Prompt for node expansion.

You are an AND/OR Tree Completeness Evaluator Agent specialized in web-browsing tasks. Your role is to assess whether a given AND node's objective has been successfully executed by the child nodes.
Node definitions:
Node Types <AND/ OR / Atomic>
- AND node: Represents an ordered list of logical sub-goals required to achieve the node's objective.
- OR node: Represents alternative sub-strategies (which can be other AND/OR nodes)
- Atomic Action: Single executable action strictly matching navigation_specifications described below.
Node status indicators:
- Unvisited nodes marked UNVISITED
- Pruned nodes marked PRUNED
- Completed nodes marked SUCCESS
- Temporarily failed nodes marked FAIL
NAVIGATION_SPECIFICATIONS:
<LIST OF BROWSER ACTIONS>
You are provided:
- The root-level task description
- Item level constraints if any
- node_id and description of the node to analyze
- Task progress summary
- Notes summary: notes taken by the agent so far - A list of its children: each with an ID, description, and status (SUCCESS, VISITED, UNVISITED, or PRUNED)
- Task feedback
Your task is to determine whether a given AND node is complete.
Follow the steps below to make this determination:
1. Examine the current children of the AND node and their statuses.
- Identify each child sub-goal or atomic action.
- Check whether each is marked as SUCCESS, FAILURE, or INCOMPLETE.
2. Review the task progress, task feedback, and notes summary.
- Look for any additional information that confirms or contradicts the completion of the node's objective.
3. Determine whether all required sub-goals or actions are present.
- Ensure no required sub-goals are missing.
- Confirm that the sub-goals appear in the correct order, as expected based on the task tree and the overall task objective.
4. Verify that each required sub-goal or atomic action is marked as SUCCESS.
- Do not assume completion unless it is explicitly indicated.
- Partial or INCOMPLETE statuses are not sufficient.
5. Conclude that the AND node is COMPLETE only if: - All required sub-goals or atomic actions are present, in the correct order, and marked SUCCESS; OR - The successful child nodes, task progress summary, and task feedback clearly indicate that the node's objective has been fully achieved.
- **You are only required to satisfy explicitly stated constraints. Do not assume or enforce any implicit constraints.**
6. Mark the AND node as INCOMPLETE if:
- The node's objective is not fully achieved, or
- Any required sub-goal, step, or relevant information is missing or not marked SUCCESS.
Special Case - Root Node Evaluation:
If the node being evaluated is the root node, you must additionally verify whether the overall task objective is fully satisfied.
- If the task requires recommending or evaluating items, then:
- You must iterate over each item that is being considered or presented as a recommendation.
- For each item, check whether it satisfies all task constraints explicitly provided in the task description or item-level constraints.
- Note that constraints that are not explicitly mentioned in the task need not be satisfied.
- The node is incomplete if any recommended item fails to meet required constraints or if evaluation is missing or partial.
- The node is incomplete if the task is incomplete and some information explicitly required by the task is missing.
- Do not assume any additional implicit constraints than the ones explicitly specified.
Begin by analyzing all the information provided (task progress summary, notes summary, node description, children description).
Provide a brief explanation for your decision, citing any missing, out-of-order, or incomplete sub-goals if relevant.
End your response in strictly one of the following formats, using a single line (reasoning AND node id must be enclosed in «»):
If node's objective is achieved:
COMPLETE «node_id»
Reasoning: «Reason why node's objective is achieved by the agent. Cite references from task progress summary, notes summary and successful children's description»
OR
If node's objective is not achieved:
INCOMPLETE «node_id»
Reasoning: «Reason why node's objective is not achieved by the agent, Cite references from task progress summary, notes summary and successful children's description»
Example 1:
COMPLETE «0.1»
Reasoning: «All required sub-goals for finding a compact digital camera with specified requirements are present, in logical order, and marked as SUCCESS.»
Example 2:
INCOMPLETE «0.2»
Reasoning: «The sub-goal to compare prices is marked as PRUNED, and the sub-goal to check reviews is missing. The AND node's objective is not fully achieved.»
You must strictly follow the above format.
{additional_context}
NODE ID : DESCRIPTION node_id : {node_description}
CHILDREN {children}

Table 18: Prompt for evaluating if an AND node's objective has been achieved.

You are a Global AND/OR Tree Update Agent revising an existing planning tree for a web-browsing task based on current available information so that the task is executed more efficiently. Do not change the ordering of the sub-plans.
Node definitions: Node Types <AND/ OR / Atomic> - AND node: Represents an ordered list of logical sub-goals required to achieve the node's objective. - OR node: Represents alternative sub-strategies (which can be other AND/OR nodes) - Atomic Action: Single executable action strictly matching navigation_specifications described below. Node status indicators: - Unvisited nodes marked UNVISITED - Pruned nodes marked PRUNED - Completed nodes marked SUCCESS - Temporarily failed nodes marked FAIL
NAVIGATION_SPECIFICATIONS: <LIST OF BROWSER ACTIONS>
You are provided: -The root-level task description -Current AND/OR tree description -HTML DOM structure as the observation - Task progress summary - Notes summary: Summary of notes taken by the agent during the task
Your task is
to carefully analyze all the information provided and determine which nodes to prune and which nodes to update.
Apply changes in this strict order:
1. PRUNE nodes that are no longer relevant or are duplicates. 2. UPDATE node descriptions if intent is unchanged but content needs minor revision.
VERY IMPORTANT RULES: - You are only allowed to PRUNE or UPDATE nodes that have not been deleted, pruned, or marked successful. - Do not add status of the node while updating the description. - Only use existing node IDs from the current tree; do not create new node IDs or subtrees. - Do not PRUNE important observations relevant to the task made in Atomic nodes with action 'note'. - Do NOT PRUNE children that are necessary for satisfying the parent node's objective. - Do not change node types. - Do not change the ordering of the sub-plans.
First reason about the update to the tree based on the information given (task description, item constraints, task progress summary, notes summary, current AND/OR tree description, observation) to you and then given your answer in the following format. Use the AND/OR tree description to ensure that you are not repeating any sub-goals that have already been considered. Format each instruction on a new single line in the format given below (All children and description must be enclosed in «». Children should be colon (;) separated):
PRUNE [node_id] UPDATE [node_id] «Describe the node's new objective (sub-goal/strategy/description of action)»
Example:
PRUNE [1.2] PRUNE [1.3] UPDATE [0.1.2] «type eggless cake in search bar» UPDATE [0.2] «Find an eggless cake recipe with over 60 votes and at least 4.5 star rating by examining search results»
Do not split the list across multiple lines.
ROOT-LEVEL TASK DESCRIPTION: {task_description}
{additional_context}
AND/OR TREE DESCRIPTION: {and_or_tree_description}
OBSERVATION: {observation}

Table 19: Prompt for global tree update.

You are a Web-browsing Agent specialized in web-browsing tasks. You have taken notes while completing a given task and now you need to generate an output for the task based on the notes you have collected. You are provided: - Task description - Sequence of notes taken during the execution of task - Planning tree representation of the task
Node definitions: Node Types <AND/ OR / Atomic> - AND node: Represents an ordered list of logical sub-goals required to achieve the node's objective. - OR node: Represents alternative sub-strategies (which can be other AND/OR nodes) - Atomic Action: Single executable action strictly matching navigation_specifications described below. Node status indicators: - Unvisited nodes marked UNVISITED - Pruned nodes marked PRUNED - Completed nodes marked SUCCESS - Temporarily failed nodes marked FAIL
Your Task is to:
Give a detailed response that directly addresses the task description, using only the information from the notes and the planning tree presentation. Do not add any additional information or context that is not present in the notes. Even if the selected notes indicate that the task was not complete or certain information is missing or constraints are not met, provide the best possible response based on the available information. Do not make assumptions or inferences beyond what is explicitly stated in the notes.
VERY IMPORTANT RULES: - Do not assume any details or context that is not explicitly mentioned in the notes.
Output Format:
Begin by analyzing the notes, then give your detailed final response in the following format.:
Task Response: «Your detailed response to the task based on the notes. If the task was not completed, this may include incomplete information or recommendations that do not meet the constraints or requirements, if applicable. If the task was completed successfully, ensure all required details are specified. Do not skip any information that is required by the task.»
Example: Task Response: «I successfully found a recipe for an eggless chocolate cake with over 60 votes and a rating of 4.5 stars. The recipe includes ingredients such as flour, sugar, cocoa powder, and baking soda. I also noted that the recipe requires 30 minutes of preparation time....»
TASK DESCRIPTION: {task_description}
NOTES TAKEN DURING TASK EXECUTION: {notes}
{additional_context}

Table 20: Prompt for generating final task response from notes taken during execution.

The framework then updates the tree structure accordingly. If the expansion fails, the agent retries the operation up to a predefined limit. After repeated failures, the agent marks the node as FAILED.

NODE REPAIR: This operator revises AND or OR nodes that have entered the FAILED state. Conditioned on the node's metadata and contextual summaries, it proposes modifications to the node's children. When the node remains repairable, the operator recommends adding new children; otherwise, it instructs the framework to prune the node. If several repair attempts fail, the agent prunes the node.

GLOBAL TREE UPDATE: This operator updates the global tree structure based on newly observed information. Given the current tree and contextual metadata, it issues instructions for pruning irrelevant or infeasible subtrees and for updating node descriptions. This operator aggressively removes unpromising branches to maintain tractability. If repeated inference attempts fail, the operator recommends fallback pruning strategies.

NODE COMPLETION CHECK: This operator evaluates whether an AND node has satisfied its objective based on the statuses of its children. The framework invokes this operator once all children have completed execution and at least one has succeeded. The operator assesses the node's descrip-

You are a Context Summarization and Critiquing Agent for web-browsing tasks.
Your job is to maintain an accurate, up-to-date understanding of task progress by analyzing:
- Task description
- Item-level constraints (if any)
- Task progress summary (if provided)
- Observation history (if provided)
- Action history (if provided)
- Notes summary: summary of notes taken by the agent so far (if provided)
- Current observation (HTML DOM)
INSTRUCTIONS:
0. Do not infer any details or make assumptions. 1. Analyze all available inputs, especially the current HTML DOM.
2. Ensure all summaries remain aligned with the main task objective.
3. Do not omit any information that may influence decisions or navigation.
4. Make summaries detailed, well-structured, and actionable.
5. Use the format below exactly.
6. Enclose the response to each section in double angle brackets: ‘« »‘.
Begin by analyzing the given context in detail. Then given your response stricly in the following format. Your output must ensure continuity, preserve all essential details, and contribute to successful task completion.
RESPONSE FORMAT:
OBSERVATION SUMMARY
«Describe the information from the CURRENT OBSERVATION. Emphasize elements and features that are relevant or potentially useful for fulfilling the task objective. Include all important detail.»
OBSERVATION HIGHLIGHTS
«Single list of integer element IDs (i.e., [123, 8765, 345]) from the current DOM that are relevant for interaction or for revisiting this step later. Only include elements from the current page. Sort by relevance, with the most important listed first.»
TASK PROGRESS
«Analyze the task progress using the task description, item-level constraints, past progress summaries, observation history, action history, and notes. First, summarize the key high-level steps the agent has actually taken toward completing the goal, focusing only on actions that were executed. Next, evaluate whether each explicitly stated task requirement or constraint has been satisfied—if the task involves item selection or recommendation, assess each item individually for compliance. Do not assume or infer unstated constraints. Finally, diagnose why the task has not yet been completed: identify any unsatisfied constraints, incorrect actions, or gaps in execution. Conclude with key takeaways or insights from the agent’s exploration that are relevant to achieving successful task completion.»
TASK FEEDBACK
«Based on the task progress summary, provide an outline of what the agent should focus on next to successfully complete the task. (2 sentences)»
Example:
OBSERVATION SUMMARY «The page shows search results for "iPhone 12 Pro Blue 128GB" on Amazon. The first relevant listing is an Apple iPhone 12 Pro, 128GB in Pacific Blue (Renewed) for $314.39. Multiple other iPhone models are also shown including iPhone 12, 13, and 14 in various colors and storage configurations....»
OBSERVATION HIGHLIGHTS
«[6028, 6033, 6042, 7204, 9277, 9280, 7242]»
NEW NOTES
«Refer to the task description and constraints, and identify all new information that can be used to completing the task. This should support future reference and continuity. Do not assume any information or make up new details. All details must be grounded in observation and actions. »
TASK PROGRESS
«The agent has successfully navigated to an Amazon search results page and identified a relevant product listing matching the task requirements. Specifically, the agent located an Apple iPhone 12 Pro with 128GB of storage in Pacific Blue, listed for $314.39 in renewed condition and fully unlocked. This matches the explicitly stated constraints for model, color, and storage capacity. The observation and notes confirm that the identified product has a strong customer rating (4.1 out of 5 from over 12,000 ratings), further supporting its relevance. All explicitly defined task constraints appear to be satisfied: model (iPhone 12 Pro), storage (128GB), color (Pacific Blue), fully unlocked, and condition (renewed). However, the task has not yet been completed. The agent has not taken action to click on the product listing, verify the full product details on the product page, or add the item to the cart. These remaining steps are essential for completing the task, as product details may vary across pages. The main takeaway is that the search and identification phase has been completed successfully, and the agent should now focus on verifying the product and adding it to the cart to fulfill the task requirements.»
TASK FEEDBACK
«The next step should be to click on the product listing to verify all details on the product page and then add it to the cart. The agent should ensure the exact model and specifications are confirmed before proceeding with the purchase. Focus on completing the final step of adding the item to cart.»
Do not output anything outside of the specified format.
TASK DESCRIPTION:
{task_description}
{additional_context}
CURRENT OBSERVATION:
{observation}

Table 21: Prompt for generating observation summary and context summary.

You are an advanced web-browsing agent that solves web-browsing related tasks.
Your job is to generate notes base on current observation as well as formulate a response to the task query based on the notes and available information.
You are given:
- Task description
- Item-level constraints (if any)
- Task progress summary (if provided)
- Action history (if provided)
- Notes: Notes taken so far (if provided)
- Current observation (HTML DOM)
INSTRUCTIONS:
0. Do not infer any details or make assumptions.
1. Analyze all available inputs, especially the current HTML DOM.
3. Do not omit any information that may influence decisions or navigation.
5. Use the format below exactly.
6. Enclose the response to each section in double angle brackets: '« »'.
Begin by analyzing the context in detail. Given your final answer strictly in the following format.
RESPONSE FORMAT:
NEW NOTES
«Refer to the task description and constraints, and identify all new information that can be used to completing the task. This should support future reference and continuity. Do not assume any information or make up new details. All details must be grounded in observation. Do not skip any important notes relevant to the task. »
TASK RESPONSE
«Give a detailed response (you may repeat information from the above notes) that directly addresses the task description and constraints (if any), using only the information from the previous notes,new notes, and the action history. Do not add any additional information or context that is not present in the available information. Even if the selected notes indicate that the task was not complete or certain information is missing or constraints are not met, provide the best possible response based on the available information. Do not make assumptions or inferences beyond what is explicitly stated in the notes.»
Example:
NEW NOTES
«Found relevant iPhone 12 Pro listing matching requirements: - Model: iPhone 12 Pro - Storage: 128GB - Color: Pacific Blue - Price: $314.39 - Condition: Renewed - Fully Unlocked - Rating: 4.1/5 stars from 12,669 ratings»
TASK RESPONSE
«Based on the task requirements, I have found an Apple iPhone 12 Pro with 128GB of storage in Pacific Blue, fully unlocked, in renewed condition, and priced at $314.39. The listing has a solid customer rating of 4.1 out of 5 from 12,669 reviews.»
Do not output anything outside of the specified format.
TASK DESCRIPTION: {task_description}
{additional_context}
PREVIOUS NOTES: {notes}
CURRENT OBSERVATION: {observation}

Table 22: Prompt for generating notes summary.

Task: Find 1 projector under $500 with 1080p resolution, Bluetooth 5.0+, ≥300 ANSI lumens brightness, and includes a 70-100" projection screen.

Item(s)	Price ($)	Native 1080p	Bluetooth ≥5.0	Brightness (≥300 ANSI)	Includes Screen (70-100")	Status	Notes
Projector A	420	Yes	Yes	350	No	To Explore	Need compatible screen under $30
Projector B + Screen Bundle	460	Yes	Yes	400	Yes	Deleted	Over budget
Projector C	390	No	Yes	500	No	Deleted	No native 1080p
Projector A + 80" Screen	445	Yes	Yes	350	Yes	Candidate	Meets all constraints

Figure 12: Example of Structured Memory used to track candidate solutions during information-seeking tasks.

tion, the task progress summary, the notes summary, and the task constraints, and returns a binary decision indicating whether the node's objective has been achieved.

OBSERVATION SUMMARIZER: This module maintains the agent's internal representations and tracks its progress. This operator processes new observations to update the task progress summary, generate an observation summary, and provide guidance for subsequent actions. When the agent encounters a new webpage, this module extracts task-relevant information, updates the agent's notes, and produces a revised notes summary. These summaries enable the agent to maintain an accurate understanding of the environment and support effective planning.

You are an advanced web-browsing agent. Your task is to maintain a table of candidate items by parsing the current webpage's HTML DOM and applying 'ADD', 'UPDATE', or 'DELETE' actions. Your goal is to keep the table accurate, up to date, and free of duplicates, based only on information explicitly visible in the DOM.
You are given:
-HTML DOM of the current webpage
- task description
- item-level constraints
VERY IMPORTANT
- Record all important details that are relevant to the task and can be used for completing the task. Include any additional relevant details.
- Only include keys whose values are **clearly visible** in the HTML DOM. Record the exact value corresponding to each key.
- Keys must match attributes listed under ITEMS AND CONSTRAINTS.
IMPORTANT RULES
Use this table to keep track of potential candidate solutions that can be used in the future for task completion or if search fails.
1. Matching and Deduplication
- Before using 'ADD', check whether the item already exists in the candidate table.
- An item is considered a duplicate if all its visible fields match an existing entry (even if the ID differs).
- If a match is found:
- Use 'UPDATE' with the existing ID.
- Only include fields that are newly visible or changed.
- Do **not** use 'ADD' for duplicates.
2. Field Visibility - Only include fields with values explicitly visible in the HTML DOM.
- Do **not** guess or infer missing values.
- Add exact values instead of simply checking if constraints is satisfied.
- Do **not** include '"unknown"' or empty-string values for missing fields.
- Omit any field that is not explicitly visible.
3. Constraint Checking
3.1 Basic Constraint Handling - For each item, evaluate which required constraints are satisfied.
- If any are violated or missing:
- Add 'constraints_not_met' listing the missing/invalid fields (e.g., 'lift range, tray type')
- Set 'status: uncertain'
- Add a brief 'comment' (e.g., "missing lift range")
3.2 Add Only When Most Constraints Are Met - Only use 'ADD' if the item satisfies **most** required constraints (at least 60%).
- If fewer than half are satisfied: - Do **not** add the item.
- If it already exists, consider using 'UPDATE' or 'DELETE'.
4. When to Use 'UPDATE'
Only use 'UPDATE' when **at least one of the following is true**:
- A previously missing field has now been found (and was not already present)
- The item now meets all required constraints and can be marked 'status: complete'
Do **not** update items:
- If nothing has changed
- If the values are already present and correct
- If the item is already marked 'deleted'
5. Deletion - Use 'DELETE' only if the item is **clearly invalid** — for example:
- It is irrelevant
- It is severely incomplete (missing most required fields)
- It does not satisfy constraints and is not fixable
- Never issue 'UPDATE' for an item that has already been deleted.
6. ID Assignment
- When using 'ADD', assign a unique ID not already present in the table (e.g., 'S102', 'CH014').
7. Item Validity
- The 'item' name must match a known type defined under ITEMS AND CONSTRAINTS.
- Skip any item with an invalid or unrecognized type.
RESPONSE FORMAT
- Return only valid 'ADD', 'UPDATE', or 'DELETE' actions — one per line.
- Do **not** return the HTML DOM or candidate table.
- Do **not** include explanation or commentary.
- If no action is needed, return an empty string.
Your response must strictly follow the following format.
Use one or more of the following actions, one per line:
ADD item:ID:UniqueID; key1:value1; key2:value2; ...
UPDATE item:ExistingID key1:value1; key2:value2; ...
DELETE item [ExistingID]
Note: item must match exactly to the items in item-level constraints.
While adding new items, always include the name or the title of the item.
EXAMPLES
ADD standing desk:ID:S102; Title:ErgoRise Desk; Price:$299.99; surface finish:Matte; status:uncertain; comment:"missing tray type and lift range"
UPDATE standing desk:S101 lift range:"24×48″"; status:complete; comment:"lift range confirmed"
DELETE standing desk [S099]
TASK DESCRIPTION {task_description}
ITEMS AND CONSTRAINTS {items_and_constraints}
CURRENT CANDIDATE TABLES {current_table}
WEB PAGE CONTENT {current_observation}

Table 23: Prompt for extracting candidate items to add to structured memory

You are a web-browsing assistant. Given a user query, extract a list of items to retrieve and, for each item, list all relevant item-specific constraints such as brand, price, rating, features, delivery time, etc.
Include any other item attributes explicitly or implicitly mentioned in the query that affect the desirability, quality, or functionality of the item (e.g., surface finish, memory presets, power type, compatibility). Treat these as additional constraints.
Include all intrinsic item-specific constraints, even if mentioned implicitly or as part of comparison criteria (e.g., "differ in brand or surface finish" implies that brand and surface finish are relevant constraints). Treat such attributes as constraints to be listed.
Ignore any source-specific constraints (e.g., "from [a specific website]") and general task-level instructions (e.g., "compare prices", "top three results"). Only include intrinsic item-specific constraints.
Only include items to retrieve—ignore any actions to perform. Ensure each item listed is unique. If there are no items to retrieve, return an empty list.
Output your response strictly in the following JSON format:
"ITEMS": ["item": "<description of item>", "constraints": ["<constraint1>", "<constraint2>", "..."] , ...]
Do not include any additional explanation. Always follow this format exactly.
TASK DESCRIPTION
{task_description}
ITEMS AND CONSTRAINTS
{items_and_constraints}
CURRENT CANDIDATE TABLES
{current_table}
WEB PAGE CONTENT
{current_observation}

Table 24: Prompt for extracting item-level constraints and attributes from task description.

You are a validation assistant for a candidate item table.
Your task is to verify whether each item satisfies the required constraints for its item type. If any constraint is violated or missing, issue an 'UPDATE' or 'DELETE' command to adjust the table accordingly.
INSTRUCTIONS
1. For each item:
- Issue an 'UPDATE' only if all of the following are true:
- The item is not marked as deleted
- One or more constraints are violated or missing
- The current values of 'constraints_not_met', 'status', or 'comment' are incorrect, incomplete, or missing
- Use the following format:
"' UPDATE item:ID constraints_not_met: <key1> <key2> ..., status: uncertain or complete; comment: "brief explanation" "'
- You are only allowed to update the following fields: - 'constraints_not_met' - 'status' - 'comment'
- Do not issue duplicate updates. Skip updates if no changes are needed.
- Issue a 'DELETE' only if:
- The item is marked as complete but fails to meet any constraint
- Or the item is mostly complete but violates most required constraints
- Use this format: "' DELETE item [ID] "'
- Do not delete items that are mostly incomplete or missing most of the required fields. These should be updated instead.
2. Item Validity - The 'item' in every 'UPDATE' or 'DELETE' command must exactly match a valid item type defined under 'ITEMS AND CONSTRAINTS'.
- Skip any action for items with invalid or unrecognized types.
3. If the item satisfies all constraints and is already marked 'status: complete', or if the item is incomplete but does not require any changes, return nothing.
RESPONSE FORMAT
- Return only 'UPDATE' and 'DELETE' commands, one per line.
- Do not return the full item rows or any explanation.
- If no action is needed, return an empty string.
- Use one or more of the following actions, one per line.
- Note: item must match exactly to the items in item-level constraints.
Your response must strictly follow the following format.
Use one or more of the following actions, one per line:
UPDATE item:ExistingID key1:value1; key2:value2; ...
DELETE item [ExistingID]
EXAMPLES
UPDATE standing desk:SD002 constraints_not_met: <lift range> <tray type>; status: uncertain; comment: "Missing lift range and tray type"
DELETE standing desk [SD010]
TASK DESCRIPTION
{task_description}
ITEMS AND CONSTRAINTS
{items_and_constraints}
CANDIDATE TABLE:
{current_table}

Table 25: Prompt for updating structured memory

You are an AND/OR Tree Repairer Agent specialized in web-browsing tasks. Your job is to repair a node in an AND/OR planning tree derived from HTML DOM structure observations.
Node definitions:
Node Types <AND/ OR / Atomic>
- AND node: Represents an ordered list of logical sub-goals required to achieve the node's objective.
- OR node: Represents alternative sub-strategies (which can be other AND/OR nodes)
- Atomic Action: Single executable action strictly matching navigation_specifications described below.
Node status indicators:
- Unvisited nodes marked UNVISITED
- Pruned nodes marked PRUNED
- Completed nodes marked SUCCESS
- Temporarily failed nodes marked FAIL
NAVIGATION_SPECIFICATIONS:
<LIST OF BROWSER ACTIONS>
You are provided:
- root-level task description
- task constraints at item level - HTML DOM structure as the observation - node_id and desription of the node to analyze - A list of its children: each with an ID, description, and status (SUCCESS, VISITED, UNVISITED, or PRUNED) - A successOR node with id and description, if present - Task progress summary of the task - Notes summary of the task : Notes taken by the agent so far
Your task:
Your task is to review all the information provided including reasoning for repair (if available) and use it to repair the node. Make sure you prune infeasible sub-goals (which have not been marked successful or pruned) and add sub-goals that are necessary for completing the node's objective:
Repair the node by pruning or adding children to the node: Remove old infeasible children and add new children that were not previously used.
VERY IMPORTANT: -Assess node repairability using node type, child status, and remaining unmet goals. -Add only new, untried children , not already present or marked SUCCESS or PRUNED. These children will be appended to existing children. -Each child must be minimal, concrete, and justified by the current DOM. -For OR nodes: Suggest only new, obvious alternatives from the DOM. -For AND nodes: Suggest only missing or incomplete sub-goals in correct order. -Do NOT PRUNE children that are necessary for satisfying the parent node's objective.
First reason about the repair and then give the final answer in the following format (Enclose all values in «» on a single line, colon (;) separated. Ordering: Prune first, then add children):
PRUNE [node_id of child] ADD [repair_node_id] repair_node_type : «1. First added sub-goal; 2. Second added sub-goal; 3. Additional added sub-goals as necessary» OR ADD [node_id] node_type : «1. Added alternative strategy 1 (score: 0.x); 2. Added alternative strategy 2 (score: 0.y); Added additional alternatives as necessary» Reasoning «Reason for the nodes pruned and added.»
Example 1: PRUNE [1246] PRUNE [1245] ADD [1244] AND : «1. Go north for 5 meters; 2. Turn left at the intersection; 3. Walk for 10 minutes»> OR ADD [1244] OR : «1. Open the door (score: 0.9); 2. Pick up the key (score: 0.8)»
Do not re-add processed or pruned children. Note that you can only prune children of the node to repair and add children to the node to repair. Added children will be appended to existing successful children. Respond using only the above. No extra text. Valid node_types are AND/OR/Atomic .
ROOT-LEVEL TASK DESCRIPTION: {task_description}
{additional_context}
OBSERVATION: {observation}
NODE ID : Description {node_id} {node_type} : {node_description}
NODE_CHILDREN_REPRESENTATION: children
REASON FOR REPAIR reason_for_repair
LOCAL TREE INFORMATION: {local_tree_info}

Table 26: Prompt for node repair

Table 27: Internal Representations Maintained by STRUCTUREDAGENT

Representation	**Description**
Interaction History	Logs all observation summaries and actions up to the current time step
Task Progress Summary	Tracks progress toward the main objective
Notes Summary	Aggregates information from all past observations that are relevant to the task
Task Constraints	Encodes global and item-level requirements from the task description
Observation Summary	Distills salient information from the current web page at each step
Action History	Lists all executed browser actions

N Human-in-the-Loop Intervention

Here we provide an extended illustration of how our framework's interpretable hierarchical structure supports human-in-the-loop oversight.

O Algorithm Details

O.1 Planning and Execution Algorithm

The agent begins by pushing the root node which represents the given task, onto the DFS stack in the *ENTERING* state. As in classical iterative DFS, the algorithm then iteratively pops the top node from the stack and processes it according to its current state.

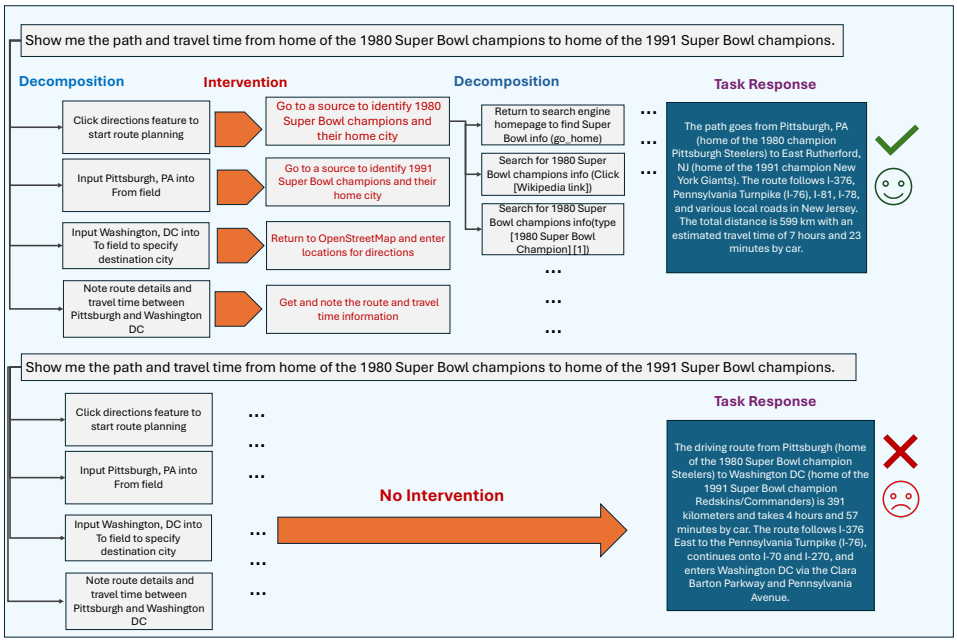


Figure 13: **Human Decomposition Intervention in AND/OR Tree Planning to Correct Bad Subtask Decomposition.** The diagram illustrates how a human decomposition intervention can redirect an AI agent that has produced an incorrect sub-goal breakdown for a multi-step task. The task requires finding the driving route between the home cities of the 1980 and 1991 Super Bowl champions. *Without intervention* (bottom), the agent incorrectly assumes destinations a priori, defaulting to Pittsburgh, PA and Washington, DC, without first verifying which teams won in those respective years, resulting in a hallucinated but confidently stated incorrect answer. *With intervention* (top), a human intercepts the agent's flawed decomposition and injects three corrective sub-goals into the AND/OR tree: (1) identify the 1980 Super Bowl champions and their home city, (2) identify the 1991 Super Bowl champions and their home city, and (3) return to the mapping tool with the verified locations. This human-guided correction grounds the plan in factual information before execution continues, yielding the correct route from Pittsburgh, PA (Pittsburgh Steelers) to East Rutherford, NJ (New York Giants) at 599 km and approximately 7 hours 23 minutes. For simplicity, we illustrate the human intervention at the first level of the AND/OR planning tree; however, such human decomposition interventions can be applied at *any level* of the tree, allowing a human operator to catch and correct errors in sub-goal formulation at whatever level of abstraction they arise.

When processing a node in the ENTERING state, the algorithm first checks the node's status. By default, new nodes are marked as unvisited. If the node is encountered for the first time, the algorithm invokes the Node Expansion module to determine its type and expand it. For AND nodes, the module generates an ordered list of children according to task constraints. For OR nodes, it ranks alternatives by their estimated likelihood of success, as determined by LLM scoring or a value function. For ACTION nodes, it outputs the precise browser command to execute. After expansion, or if the node was previously expanded, the agent marks the node as visited and re-adds it to the stack in the *EXITING* state.

Next, the agent proceeds according to node type. For AND nodes, it pushes each child onto the stack in the ENTERING state for further processing. For OR nodes, it pushes only the most promising unprocessed child in the ENTERING state. If the node has no children, no further action is taken until its status is evaluated in the EXITING state. For ACTION nodes, the agent immediately executes the specified browser command. If the command is successful, the agent marks the node as successful, updates its internal context via the Observation and Notes Summarization module. The Global Tree Update module prunes irrelevant branches and refines node descriptions as needed, propagating these changes upward to keep the stack and tree consistent. If the ACTION node fails, the agent marks it as *FAILED*.

Algorithm 2: PROCESS_NODE_ENTERING(node, stack)

```
// Perform rollback when parent is an OR node
if node.parent and node.parent.type == NodeType.OR then
    success ← PERFORM_ROLLBACK(node.parent)
    if success == true then
        currentUrl ← node.parent.url
    end
end
// Increase node execution count
node.execution_count ← node.execution_count + 1
// If node hasn't yet been visited, expand it
if node.type == NodeType.UNKNOWN and node.status ∉ CLOSED_STATUSES then
    // Populate the node type
    node, _, ACTION ← POPULATE_NODE_TYPE(node)
end
node.status ← NodeStatus.VISITED
// Process node based on its type
if node.type == NodeType.ACTION then
    stack, node ← PROCESS_ACTION_NODE(node, stack)
                                                                    // Algorithm 6
end
else if node.type == NodeType.AND then
    stack, node ← PROCESS_AND_NODE(node, stack)
                                                                    // Algorithm 7
end
else if node.type == NodeType.OR then
    stack, node ← PROCESS_OR_NODE(node, stack)
                                                                    // Algorithm 8
end
return stack, node
```

When processing a node in the EXITING state, the agent evaluates completion criteria based on the node type. For AND nodes, the agent verifies that all valid, that is, unpruned or undeleted, child nodes have succeeded and invokes the Node Completion module to ensure that the overall objective of the AND node is met. Although this check is theoretically applicable to all AND nodes, we found it to be overly strict in practice. Therefore, to avoid unnecessary retries and failures, we apply this strict completion check only at the root node in our implementation. If completion is verified, the agent marks the node as successful, otherwise, as *FAILED*. For OR nodes, the agent checks if at least one child has succeeded, marking the node as *SUCCESS* if so, and as *FAILED* otherwise. For ACTION nodes, the agent sets the status to success or *FAILED* depending on the outcome of the browser operation. Once a node is marked as success, it is not processed again.

If the agent processes a node in the FAILED state, it attempts to repair the node or marks it as pruned. For ACTION nodes, it prunes the node and updates the stack. If a failed ACTION node belongs to an AND node, the agent deletes all remaining unexecuted siblings, since the conjunctive requirement cannot be satisfied. For AND nodes, if none of the children have achieved the node's objective, the agent checks the node's revision count. If the revision limit is not reached, the Node Repair module attempts to generate new sub-goals or modify the set of children. If repair fails or the revision limit is reached, the agent prunes the AND node and all its descendants, propagating failure upward and updating the stack. For OR nodes in the FAILED state, the agent first checks for any remaining unprocessed children. If such children exist, it selects and pushes the most promising one onto the stack in the ENTERING state. If all alternatives are exhausted, it attempts node repair if within the revision limit, otherwise, it prunes the node. As with AND nodes, any pruning or failure of an OR node propagates upward, ensuring the global plan remains consistent and that failures at lower levels trigger repair or pruning at higher levels.

This DFS-based algorithm allows the agent to incrementally build and adapt its hierarchical plan, efficiently responding to new observations and failures, while executing the task.

Algorithm 3: PROCESS_NODE_EXITING(node, stack)

```
// ACTION NODE IN EXITING STATE
if node.type == NodeType.ACTION then
    // Put them back into the stack in failure mode
    if node.status ∈ FAILED_OR_PRUNED then
        stack.append((node, NodeState.FAILED))
    end
    else
        node.status ← NodeStatus.SUCCESS
    end
end
// AND NODE IN EXITING STATE
else if node.type == NodeType.AND then
    success ← IS_SUCCESSFUL_AND(node)
    if success == true then
        // If node revision has not reached max limit, check if node is complete
        // If not complete, repair the node
        if (node.revision_count < MAX_REVISION_COUNT) then
            isComplete, reasoning ← CHECK_FOR_COMPLETION_AND(node)
            node.reasoning ← reasoning
            if isComplete == true then
                node.status ← NodeStatus.SUCCESS
                continue
            end
            else
                // Is complete check failed, mark node as failed and push it on the stack in FAIL
                   state
                node.status ← NodeStatus.FAIL
                node.reasoning ← reasoning
                stack.append((node, NodeState.FAILED))
                continue
            end
        end
        else
            // If node has already been revised sufficient number of times and the children are
               successful, mark the node as successful
            node.status ← NodeStatus.SUCCESS
            continue
        end
    end
    else
        // Mark the node as failed and push it onto the stack in FAIL state
        node.status ← NodeStatus.FAIL
        stack.append((node, NodeState.FAILED))
    end
end
// OR NODE IN EXITING STATE
else if node.type == NodeType.OR then
    success ← IS_SUCCESSFUL_OR(node)
    if success == true then
        node.status ← NodeStatus.SUCCESS
    end
    else
        node.status ← NodeStatus.FAIL
        stack.append((node, NodeState.FAILED))
    end
end
return stack, node
```

Internal Functions Reference

Node Status and Type Collections

P Planning Tree

Algorithm 4: PROCESS_NODE_FAILED(node, stack)

```
// ACTION NODE IN FAILED STATE
if node.type == NodeType.ACTION then
    node.status ← NodeStatus.PRUNED
    BACKTRACK_FAILURE(node)
end
// Process node based on its type
else if node.type == NodeType.AND then
    stack, node ← PROCESS_FAILED_AND_NODE(node, stack)
                                                                    // Algorithm 9
end
else if node.type == NodeType.OR then
    stack, node ← PROCESS_FAILED_OR_NODE(node, stack)
                                                                    // Algorithm 10
end
return stack, node
```

Algorithm 5: BACKTRACK_FAILURE(node)

```
if node.parent == null then
    return
                                                                    // No parent to backtrack to
end
if node.parent.type == NodeType.AND and node.parent.ordered == true then
    // Handle ordered AND node: delete remaining siblings
    remaining ← GET_REMAINING_EXCLUDING_NODE(node, node.parent)
    undeletedChildren ← []
    foreach rem in remaining do
        if rem.status != NodeStatus.DELETED then
            undeletedChildren.append(rem)
                                                                    // Filter undeleted children
        end
    end
    if undeletedChildren not empty then
        foreach rem in undeletedChildren do
            rem.status ← NodeStatus.DELETED
                                                                    // Mark as deleted
        end
        deletedChildren ← RECURSIVELY_DELETE_CHILDREN(undeletedChildren)
        if stack not empty then
            stack ← PURGE_STACK(deletedChildren, stack)
                                                                    // Remove from stack
        end
    end
end
```

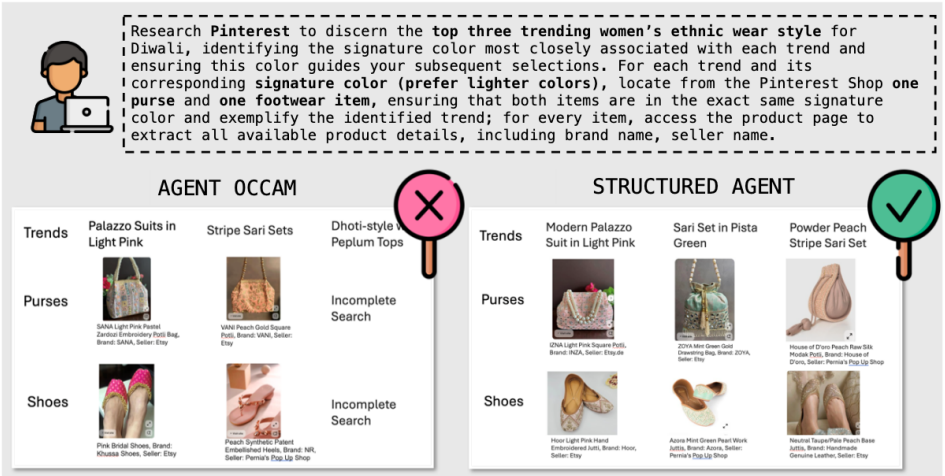


Figure 14: AgentOccam vs STRUCTUREDAGENT on a Pinterest task: AgentOccam terminates its search prematurely when it fails to explore the third trend. On the other hand, STRUCTUREDAGENT is able to successfully retrieve items for all three trending women's ethnic wear styles.

Algorithm 6: PROCESS_ACTION_NODE(node, stack)

```
// If node represents an ACTION, execute it. If successful, globally update tree
status ← null
// Before processing any node, push it back onto stack in EXITING state
stack.append((node, NodeState.EXITING))
success ← false
// Execute action with retry logic
while node.retry_count < MAX_RETRY_COUNT do
    // Try block for attempting to perform action
    if NO_EXCEPTION then
        action, status ← PERFORM_ACTION(node)
        if env.steps > MAX_STEPS then
            return stack, node                                          // Reached maximum steps
        end
        if env.done() then
            return stack, node                                          // Environment done after performing action
        end
        // Check if action was successfully executed
        assert(status is True)
        // Extract notes from the action if there are any
        UPDATE_NOTES(node.action)
        // Mark ACTION node as successful
        node.status ← NodeStatus.SUCCESS
        success ← true
        // If ACTION has resulted in change inn current observation
        isNoteAction ← IS_NOTE_ACTION(node.action)
        if not isNoteAction then
            // Update notes and observation summaries
            ASYNC_FULL_UPDATE()
            STEP ← STEP + 1
            // Update URL of current node and current URL
            currentUrl ← env.get_url()
            UPDATE_URL(node)
        end
        node.retry_count ← node.retry_count + 1
        break
    end
    // If exception occurred, update retry count and continue
    node.retry_count ← node.retry_count + 1
end
// If success is false, update status to fail
if success == false then
    node.status ← NodeStatus.FAIL
end
return stack, node
```

Algorithm 7: PROCESS_AND_NODE(node, stack)

```
// For an AND node, process all unprocessed children and put them on stack
// First push the node back onto stack in EXITING state
// so that control returns to it once children have been executed
stack.append((node, NodeState.EXITING))
children ← node.children
// Check if the AND node is valid
isValid ← IS_VALID_AND(children)
// Check if the AND node is successful
success ← IS_SUCCESSFUL_AND(node)
// If successful, mark node as successful
if success then
    node.status ← NodeStatus.SUCCESS
end
else if not isValid then
    // I there are no valid children, mark it as fail
    node.status ← NodeStatus.FAIL
end
else
    // Put unprocessed children in stack in reversed order
    foreach child in reverse(node.children) do
        if child.status ∉ CLOSED_STATUSES then
            stack.append((child, NodeState.ENTERING))
        end
    end
end
return stack, node
```

Algorithm 8: PROCESS_OR_NODE(node, stack)

```
// For an OR node, process next promising child
// First push node back onto stack in EXITING state
// to return control to it after child has been processed
stack.append((node, NodeState.EXITING))
children ← node.children
// Check if node is successful
success ← IS_SUCCESSFUL_OR(node)
// There should be one valid child
isValid ← IS_VALID_OR(children)
// Mark the node as successful
if success then
    node.status ← NodeStatus.SUCCESS
end
// If no children are available to process, mark node as failed
else if not isValid then
    node.status ← NodeStatus.FAIL
end
else
    // Find next promising child (children ordered by estimated likelihood of success)
    child ← FIND_NEXT_PROMISING(children)
    // Push promising child onto the stack
    stack.append((child, NodeState.ENTERING))
end
return stack, node
```

Algorithm 9: PROCESS_FAILED_AND_NODE(node, stack)

```
// For an AND node in FAILED state, revise, prune, or backtrack
// Initialize tracking variables
delChildren ← []
addedChildren ← []
prunedChildren ← []
revised ← false
isValid ← IS_VALID_AND(node.children)
// If node is invalid but has at least one successful child, check if the node's objective has been
   completed
if not isValid then
    reasoning ← ""
    hasOneSuccess ← HAS_ATLEAST_ONE_SUCCESS(node)
    if hasOneSuccess then
        isComplete, reasoning ← CHECK_FOR_COMPLETION_AND(node)
        node.reasoning ← reasoning
        if isComplete and isComplete == true then
            node.status ← NodeStatus.SUCCESS
            return stack, node
        end
    end
    // If node revision has not reached the maximum limit, repair it
    node.status ← NodeStatus.FAIL
    if (len(node.children) < MAX_CHILDREN and node.revision_count < MAX_REVISION_COUNT) then
        // Revise the node
        node, delChildren, prunedChildren, addedChildren ← REVISE_AND_NODE(node)
        removedChildren ← delChildren + prunedChildren
        // Update stack and purge all the removed children
        stack ← PURGE_STACK(removedChildren, stack)
        // Check if node has been revised. If revised, update revision count
        if node.status != NodeStatus.PRUNED and (len(addedChildren) != 0 or len(prunedChildren) != 0) then
            revised ← true
            node.revision_count ← node.revision_count + 1
        end
    end
end
// If node is successful, mark the node as successful
if node.status == NodeStatus.SUCCESS then
    RECURSIVELY_MARK_SUCCESS(node.id)
end
// If the node has been revised or there are children to be processed, then push the node back onto
   the stack in ENTERING state
else if revised or isValid then
    node.status ← NodeStatus.VISITED
    stack.append((node, NodeState.ENTERING))
end
// If node is invalid and not revised, prune the node
else if not revised and not isValid then
    prunedNodes ← RECURSIVELY_PRUNE(node.id)
    // Update stack and purge all the pruned nodes
    stack ← PURGE_STACK(prunedNodes, stack)
    // Propagate failure
    BACKTRACK_FAILURE(node)
end
return stack, node
```

Algorithm 10: PROCESS_FAILED_OR_NODE(node, stack)

```
// If an OR node is in failed state, either execute the next promising child
// (if any) or repair or prune the node. If node is pruned and the node's parent is AND,
// then purge the rest of the children, or else if the parent is OR, then process the next promising
   child
revised ← false
addedChildren ← []
isValid ← IS_VALID_OR(node.children)
if isValid then
    node.status ← NodeStatus.VISITED
    stack.append((node, NodeState.ENTERING))
    return stack, node
end
else
    // If the node revision count has not reached the max limit, revise the node
    if node.revision_count < MAX_REVISION_COUNT then
        // Revise OR node
        node, addedChildren, prunedChildren ← REVISE_OR_NODE(node)
        // Update stack
        stack ← PURGE_STACK(prunedChildren, stack)
        // Check if node was revised, update revision count
        if len(addedChildren) > 0 then
            revised ← true
            node.revision_count ← node.revision_count + 1
        end
    end
    // If node was revised, push it back onto the stack in ENTERING STATE
    if revised then
        node.status ← NodeStatus.VISITED
        // If new children were added, put this node back onto the stack
        stack.append((node, NodeState.ENTERING))
    end
    else
        // If not revised then prune the node
        prunedNodes ← RECURSIVELY_PRUNE(node.id)
        // Update stack
        stack ← PURGE_STACK(prunedNodes, stack)
        // Propagate failure and process next promising child of OR node
        BACKTRACK_FAILURE(node)
    end
end
return stack, node
```

Table 28: Node Status Definitions in STRUCTUREDAGENT Framework

Status	Description
UNVISITED	Initial state of all nodes before processing begins.
VISITED	Node has been processed but final outcome is not yet determined.
SUCCESS	Node's objective is achieved. For AND nodes, all children succeed and collectively satisfy the objective; for OR nodes, any single child's success suffices; for ACTION nodes, the corresponding browser operation succeeds.
FAIL	Node's objective cannot be achieved. For AND nodes, occurs if any child fails or is pruned; for OR nodes, if the selected child fails; for ACTION nodes, if the operation is invalid or unsuccessful.
PRUNED	Node becomes irrelevant or irresolvable due to new information discovered during execution.
DELETED	Node is marked as inapplicable due to failure or pruning of a predecessor sibling in an AND node with ordering constraints.

Table 29: Internal Functions Reference

Function Name	Description
PERFORM_ROLLBACK(node)	Performs context rollback to restore the browser state to the URL of the given node. Returns success status. Used when processing OR node children to ensure clean state.
POPULATE_NODE_TYPE(node)	Expands an UNKNOWN node by determining its type (ACTION, AND, OR) and populating relevant fields like children or action details. Returns updated node, status, and action.
BACKTRACK_FAILURE(node)	Handles backtracking when a node fails. For ordered AND nodes, deletes remaining siblings and their descendants. Propagates failure up the tree.
PERFORM_ACTION(node)	Executes the action specified in the node (e.g., click, type, navigate). Interacts with the environment to perform the web automation task. Returns action and execution status.
UPDATE_NOTES(action)	Extracts and updates notes from the action if any are present. Notes are used to record information during execution.
IS_NOTE_ACTION(action)	Checks if an action is a "note" action. Note actions don't change observations, so they don't require a full tree update.
ASYNC_FULL_UPDATE()	Performs a global asynchronous update of the agent's state, observation summaries, and tree structure after an observation-changing action.
UPDATE_URL(node)	Updates the URL field of the given node to match the current environment URL.
IS_VALID_AND(children)	Checks if an AND node is valid by verifying it has at least one non-closed child that can be processed.
IS_SUCCESSFUL_AND(node)	Checks if an AND node is successful by verifying all its children have succeeded.
CHECK_FOR_COMPLETION_AND(node)	Evaluates whether an AND node's objective has been completed even if not all children succeeded. Returns completion status and reasoning. Used to determine if partial completion is acceptable.
HAS_ATLEAST_ONE_SUCCESS(node)	Checks if a node has at least one successful child. Used to determine if completion checking should be performed for AND nodes.
IS_SUCCESSFUL_OR(node)	Checks if an OR node is successful by verifying at least one of its children has succeeded.
IS_VALID_OR(children)	Checks if an OR node is valid by verifying it has at least one child available to process.
FIND_NEXT_PROMISING(children)	Finds the next most promising child to explore from a list of OR node children. Children are ordered by estimated likelihood of success.
REVISE_AND_NODE(node)	Revises a failed AND node by analyzing failures, potentially pruning unsuccessful children, and adding new children. Returns updated node, deleted children, pruned children, and added children.
REVISE_OR_NODE(node)	Revises a failed OR node by generating new alternative approaches. Returns updated node, added children, and pruned children.
GET_REMAINING_EXCLUDING_NODE(node, parent)	Returns all sibling nodes that come after the given node in the parent's children list. Used for deleting remaining children in ordered AND nodes.
RECURSIVELY_DELETE_CHILDREN(children)	Marks the given children and all their descendants as deleted in the tree structure. Returns list of deleted nodes.
RECURSIVELY_PRUNE(node_id)	Recursively prunes a node and all its descendants, marking them as pruned in the tree structure. Returns list of pruned nodes.
RECURSIVELY_MARK_SUCCESS(node_id)	Recursively marks a node and its relevant descendants as successful. Used to propagate success status through the tree.
PURGE_STACK(deletedChildren, stack)	Removes all deleted or pruned nodes from the execution stack to prevent processing invalid branches. Returns updated stack.

Table 30: Node Status and Type Collections

Collection	Description
CLOSED_STATUSES	Node statuses that indicate the node is no longer active or executable: [SUCCESS, PRUNED, DELETED]
FAILED_STATUSES	Node statuses that imply failure or deleted: [PRUNED, DELETED]
FAILED_OR_PRUNED	Node statuses that indicate failure or pruned: [FAIL, PRUNED]
VISITED_TYPES	Node statuses that indicate a node has been visited or processed in some way: [FAIL, VISITED, PRUNED, SUCCESS]
VALID_NODE_TYPES	All valid/expected node types (excluding UNKNOWN): [AND, OR, ACTION]
NON_LEAF_NODES	Node types that represent composite (non-leaf) logic structures: [AND, OR]

Table 31: AND/ORTree Node Attributes

Attribute	Type	Description
type	NodeType	The type of the node (e.g., AND, OR, ACTION)
description	str	A human-readable description of the node
id	str	A unique identifier for the node
steps	int	Number of steps or operations this node represents (default: 0)
children	List[Node]	A list of child nodes (default: empty list)
parent	Optional[Node]	A reference to the parent node, if any
status	NodeStatus	The current execution status of the node (default: UNVISITED)
execution_count	int	Number of times this node has been executed (default: 0)
retry_count	int	Number of retry attempts for this node (default: 0)
revision_count	int	Number of revisions or changes to this node (default: 0)
depth	int	Depth of the node within the tree (root is 0, default: 0)
ordered	bool	Whether the execution of child nodes is ordered (default: True)
url	str	Optional URL associated with the node (default: empty string)
action	str	Action or command associated with the node (default: empty string)
reasoning	str	Explanation or rationale behind this node's inclusion or action (default: empty string)
others	dict	Arbitrary additional metadata or context (default: empty dict)
notes	List[str]	Freeform notes or comments related to the node (default: empty list)

- [0] (AND) Research Pinterest to identify the top 3 women's ethnic wear trends for Diwali. For each trend: 1) Specify the signature color currently trending. 2) Find one matching footwear style and one purse, both in the same color (same color as the trend) from the Pinterest Shop. 3) Provide the name of a potential brand or seller for each footwear and purse, and include a direct source or pin link if possible. Focus on vibrant, festive, and popular looks. Do not end your search early. Limit your review to only the first 2 scrolls of results. Do not click on external links. [success]
- [0.1] (Atomic) Search for Diwali ethnic wear trends on Pinterest (type [401] [Diwali ethnic wear trends 2024] [1]) [success]
- [0.2] (Atomic) Document 3 trending ethnic wear styles with their signature elements (note [Trending Diwali Ethnic Wear 2024: 1. Pink - Anupa Sari with embellished blouse from INDIASPOPUP.COM 2. Pista Green - Crush fabric sari with sequins embroidery border and yellow jacquard blouse from Etsy 3. Powder Peach - Stripe sari set with thread, pearl and sequin embroidery, featuring plunging neckline blouse from KYNAH]) [success]
- [0.3] (AND) Navigate to Pinterest Shop and search for footwear in Pink, Pista Green, and Powder Peach shades from verified sellers specifically for Diwali looks [success]
- [0.3.1] (Atomic) Navigate to Pinterest Shop section (click [208]) [success]
- [0.3.2] (Atomic) type [401] [Pink ethnic juttis Diwali footwear] [1] (type [401] [Pink ethnic juttis Diwali footwear] [1]) [success]
- [0.3.3] (Atomic) Return to previous page after completing pink footwear search (go_back) [success]
- [0.3.4] (Atomic) type [401] [Pista Green ethnic juttis Diwali] [1] (type [401] [Pista Green ethnic juttis Diwali] [1]) [success]
- [0.3.5] (Atomic) Return to previous page to continue accessory search (go_back) [success]
- [0.3.6] (Atomic) type [401] [Powder Peach ethnic juttis Diwali] [1] (type [401] [Powder Peach ethnic juttis Diwali] [1]) [success]
- [0.4] (OR) Search Pinterest Shop for matching ethnic clutches/purses in three colors by trying alternative search terms: 1) Search "Pink potli Diwali bag" or "Pink ethnic clutch" from verified sellers, 2) Search "Pista Green potli Diwali bag" or "Mint ethnic clutch" from verified sellers, 3) Search "Peach potli Diwali bag" or "Blush ethnic clutch" from verified sellers. Limit to first 2 scrolls of results. [success]
- [0.4.1] (AND) Search for powder peach ethnic clutches in Shop section [success]
- [0.4.1.1] (Atomic) Search for powder peach ethnic clutches in Shop section (type [401] [powder peach ethnic clutch Diwali] [1]) [success]
- [0.4.1.2] (Atomic) Execute search query for powder peach ethnic clutches (type [401] [powder peach ethnic potli clutch bag Diwali] [1]) [success]
- [0.4.1.4] (Atomic) note [Found matching powder peach potli bags from verified sellers: 1. 5 Elements Peach Brocade Pattern Potli Bag at Pernia's Pop Up Shop 2. AMYRA Peach Poly Silk Pearl Embroidered Potli Bag 3. SWABHIMANN ACCESSORIES Peach Pure Silk Potli Bag] (note [Found matching powder peach potli bags from verified sellers: 1. 5 Elements Peach Brocade Pattern Potli Bag at Pernia's Pop Up Shop 2. AMYRA Peach Poly Silk Pearl Embroidered Potli Bag 3. SWABHIMANN ACCESSORIES Peach Pure Silk Potli Bag]) [success]
- [0.4.1.5] (Atomic) Modify search query to find matching powder peach potli bags (type [401] [powder peach potli bag Diwali] [1]) [success]
- [0.4.1.6] (Atomic) click [208] to ensure Shop filter is active before searching for powder peach accessories (click [208] to ensure Shop filter is active before searching for powder peach accessories) [success]
- [0.4.1.8] (Atomic) Modify search to find powder peach potli bags for Diwali (type [401] [powder peach potli bag Diwali] [1]) [success]
- [0.5] (Atomic) Document powder peach accessory pairs with seller details (note [Powder Peach - Footwear: Coral Haze Juttis, Jutti Express formal wear, 5 Elements ethnic shoes; Purse: 5 Elements Brocade Potli with tassels, AMYRA Poly silk Pearl Embroidered Potli, Ri Ritu Kumar Net and Silk Zari Potli]) [success]
- [0.6] (AND) Document pink and pista green accessory pairs with seller details and pin links, ensuring color coordination and authenticity of sellers [success]
- [0.6.1] (AND) Search and document pink ethnic footwear with seller details [success]
- [0.6.1.1] (Atomic) Search for pink ethnic footwear for Diwali (type [401] [pink ethnic juttis Diwali] [1]) [success]
- [0.6.1.2] (Atomic) note [Found matching pink footwear options: 1. B Anu Designs - Baby Pink Punjabi Jutti with Mirror and Bead handwork 2. Miar Designs - Pink Leather Embroidered Juttis at Pernia's Pop Up Shop 3. YASSIO - Pink Silk Floral Embroidered Juttis] (note [Found matching pink footwear options: 1. B Anu Designs - Baby Pink Punjabi Jutti with Mirror and Bead handwork 2. Miar Designs - Pink Leather Embroidered Juttis at Pernia's Pop Up Shop 3. YASSIO - Pink Silk Floral Embroidered Juttis]) [success]
- [0.6.1.3] (Atomic) Return to previous page to continue accessory search (go_back) [success]
- [0.6.2] (Atomic) Document matching pink potli/clutch from search results (note [Found matching pink brocade potli bag from 5 Elements with tassels and beads embellishments, suitable for festive wear]) [success]
- [0.6.3] (Atomic) Return to previous page after powder peach potli search (go_back) [success]
- [0.6.4] (Atomic) Search for pista green ethnic footwear using specific search terms (type [401] [Pista green embroidered juttis Diwali] [1]) [success]
- [0.6.5] (Atomic) Search for matching pista green clutch/potli (type [401] [Pista green brocade potli Diwali] [1]) [success]
- [0.6.6] (Atomic) Document complete accessory details with seller information for pista green trend accessories: Footwear - House of Vian's Crepe Embroidered Juttis and 5 Elements' Leatherette Embellished Juttis; Purse - 5 Elements Green Brocade Embroidered Potli with tassels and dupion lining, and Tarini Nirula Green Brocade Stone Embellished Potli with antique gold frame (Document complete accessory details with seller information for pista green trend accessories: Footwear - House of Vian's Crepe Embroidered Juttis and 5 Elements' Leatherette Embellished Juttis; Purse - 5 Elements Green Brocade Embroidered Potli with tassels and dupion lining, and Tarini Nirula Green Brocade Stone Embellished Potli with antique gold frame) [success]
- [0.6.7] (Atomic) Search for powder peach ethnic footwear (type [401] [powder peach ethnic juttis Diwali] [1]) [success]
- [0.6.10] (AND) Search for pink ethnic footwear pins from verified sellers and save direct links [success]
- [0.6.10.1] (Atomic) View details of pink leather embroidered juttis for Pink category (click [17999]) [success]
- [0.6.10.2] (Atomic) note [Pink leather juttis by Miar Designs - Features: Leather base with coin, thread, pearl, sequins and beads embroidery, Available at Pernia's Pop Up Shop, Price: 40,00, Formal occasion wear] (note [Pink leather juttis by Miar Designs - Features: Leather base with coin, thread, pearl, sequins and beads embroidery, Available at Pernia's Pop Up Shop, Price: 40,00, Formal occasion wear]) [success]
- [0.6.10.3] (Atomic) Return to previous page to continue purse search (go_back) [success]
- [0.6.11] (Atomic) Search for matching pink potli/clutch pins (type [401] [pink ethnic potli clutch Diwali] [1]) [success]
- [0.6.14] (Atomic) Verify and document color coordination between footwear-purse pairs (note [Color coordination verified: Pink category - Pink Beaded Mirror Work Potli and Pink Brocade Potli perfectly match the pink leather juttis from Miar Designs in shade and festive style]) [success]
- [0.7] (Atomic) Document direct pin links for pink accessories from verified sellers (note [Found verified seller pin links: 1. Pink Brocade Tassels Embellished Potli by 5 Elements - Direct Pin ID 20730 2. Pink Brocade Ghungroo Embroidered Potli by 5 Elements - Direct Pin ID 20752 3. Pink Beaded Mirror Work Potli by Etsy verified seller - Direct Pin ID 21242]) [success]
- [0.8] (AND) Review and document direct pin links for all three color categories ensuring each has verified footwear and purse pairs with seller details [success]
- [0.8.1] (Atomic) Document completed findings for Pink category (note [Pink Category Complete - Footwear: Miar Designs, Purses: Pink Beaded Mirror Work Potli (Etsy), Pink Brocade Potli (5 Elements)]) [success]
- [0.8.2] (Atomic) Navigate to Pinterest Shop section for ethnic potli clutches (click [20411]) [success]
- [0.8.7] (Atomic) Search for powder peach potli options (type [401] [powder peach ethnic potli clutch Diwali] [1]) [success]
- [0.8.8] (Atomic) Search for matching Pista Green potli options with verified sellers and clear product details (type [401] [pista green ethnic potli clutch Diwali] [1]) (Search for matching Pista Green potli options with verified sellers and clear product details (type [401] [pista green ethnic potli clutch Diwali] [1])) [success]
- [0.8.9] (Atomic) Document verified seller information for Pista Green potlis (note [Verified Pista Green Potlis: 1. PLODE - Green Velvet Swarovski Triangle Potli with crystal embellishments and rhinestone strap 2. Velleso - Green Mirror Work Potli with crystal drops and silver threadwork (24x23cm) 3. The Pink Potli - Green Raw Silk Potli with bead work and tassel embellishments]) [success]
- [0.8.10] (Atomic) Search for matching Powder Peach ethnic footwear options (type [401] [powder peach ethnic footwear Diwali] [1]) [success]
- [0.8.11] (Atomic) Search for matching Pista Green ethnic footwear options with verified sellers and clear product details (type [401] [pista green ethnic footwear Diwali] [1]) [success]
- [0.8.12] (Atomic) Document pin links and details for Pista Green category footwear and purse pairs (note [Found direct pin links for Pista Green category: 1) House of Vian Green Crepe Embroidered Juttis with crystals and gota patti from Pernia's Pop Up Shop 2025, 2) 5 Elements Green Leatherette Embellished Juttis with beadwork from Pernia's Pop Up Shop 2025]) [success]

Table 32: Planning tree for the task *"Research Pinterest to identify the top 3 women's ethnic wear trends for Diwali. For each trend: 1) Specify the signature color currently trending. 2) Find one matching footwear style and one purse, both in the same color (same color as the trend) from the Pinterest Shop. 3) Provide the name of a potential brand or seller for each footwear and purse, and include a direct source or pin link if possible. Focus on vibrant, festive, and popular looks. Do not end your search early. Limit your review to only the first 2 scrolls of results. Do not click on external links."*

Table 33: Trajectory and final response comparison on task **Diwali-05**: *"Research Pinterest to identify the top 3 women's ethnic wear trends for Diwali. For each trend: 1) Specify the signature color currently trending. 2) Find one matching footwear style and one purse, both in the same color (same color as the trend) from the Pinterest Shop. 3) Provide the name of a potential brand or seller for each footwear and purse, and include a direct source or pin link if possible. Focus on vibrant, festive, and popular looks. Do not end your search early. Limit your review to only the first 2 scrolls of results. Do not click on external links."* Both agents use `claude-3-5-sonnet-20241022`.

Step	**StructuredAgent**	**AgentOccam**
1	`search` "Diwali ethnic wear trends 2024"	`search` "Diwali ethnic wear trends 2024"
2	`click` search bar	`click` first result
3	`search` "light pink ethnic purse palazzo style"	`click` search bar
4	`click` product result	*[note]* Top 3 trends: (1) Palazzo suits – Light Pink, (2) Stripe sari sets – Powder Peach, (3) Dhoti-style with peplum tops
5	`go_back`	`search` "light pink purse ethnic"
6	`search` "light pink traditional juttis wedding"	`click` product result
7	`click` product result	`search` "light pink ethnic sandals heels"
8	`go_back`	`search` "light pink ethnic sandals heels" (retry)
9	`search` "pista green traditional potli clutch"	*[note]* Purse: SANA Light Pink Zardozi Potli, $45.00 (Etsy)
10	`search` "pista green ethnic potli bag traditional" (retry)	`search` "light pink ethnic sandals heels" (retry)
11	`go_back`	`search` "light pink ethnic sandals heels" (retry)
12	`search` "mint green ethnic jutti traditional"	`go_back`
13	`go_back`	`click` search bar
14	`click` product result	`click` search bar
15	`search` "mint green ethnic bags wedding"	`search` "light pink ethnic heels sandals"
16	`search` "mint green ethnic footwear wedding"	`click` product result
17	`search` "peach traditional clutch ethnic"	*[note]* Footwear: Pink Bridal Juttis, $63.99 (Etsy)
18	`search` "peach traditional jutti wedding"	`search` "powder peach ethnic purse"
19	—	`click` product result
20	—	*[note]* Purse: VANI Peach Gold Potli, $45.00 (Etsy)
21	—	`search` "powder peach ethnic sandals heels"
22	—	`click` product result
23	—	*[note]* Footwear: Peach Embellished Heels, $49.00 (Pernia's)
24	—	`search` "ethnic purse for dhoti style"
25	—	`stop` (explicit failure on trend 3)
Final Response		
Trend 1	**Light Pink Palazzo Suit.** *Bag:* IZNA Light Pink Square Potli, $45.00 (Etsy.de); zardozi embroidery, pearl handle. *Footwear:* Hoor Light Pink Hand Embroidered Jutti, $39.51 (Etsy).	**Light Pink Palazzo Suit.** *Bag:* SANA Zardozi Potli, $45.00 (Etsy). *Footwear:* Pink Bridal Juttis, $63.99 (Etsy).
Trend 2	**Pista Green Crush Fabric Sari Set.** *Bag:* ZOYA Mint Green Gold Drawstring Bag, $57.50 (Etsy); mirror embroidery. *Footwear:* Azora Mint Green Pearl Work Juttis (Pernia's Pop-Up Shop).	**Powder Peach Stripe Sari Set.** *Bag:* VANI Peach Gold Potli, $45.00 (Etsy). *Footwear:* Peach Embellished Heels, $49.00 (Pernia's Pop-Up Shop).
Trend 3	**Powder Peach Stripe Sari Set (KYNAH).** *Bag:* House of D'oro Peach Raw Silk Modak Potli, $59.00 (Pernia's). *Footwear:* Pale Peach Handcrafted Punjabi Jutti, $75.00 (Etsy).	**Dhoti-style with Peplum Tops.** Accessories not found; failure explicitly acknowledged via `stop`.